

# zk-SNARKs for First Order Logic

Murdoch J. Gabbay\*\*  
Heriot-Watt University  
Edinburgh, United Kingdom  
[m.gabbay@hw.ac.uk](mailto:m.gabbay@hw.ac.uk)

Andrew Mendelsohn†  
Imperial College London  
London, United Kingdom  
[am3518@ic.ac.uk](mailto:am3518@ic.ac.uk)

## Abstract

We efficiently arithmetise first order logic (FOL) to zk-SNARK-friendly relations. FOL is a highly expressive and widely-used mathematical logic capable (amongst other things) of expressing Turing-complete computational systems. We provide a framework for arithmetising FOL by extending a univariate polynomial semantics for FOL (J. of Applied Logics '25) to a general semantics parameterised by a chosen tuple of functions satisfying some compatibility properties. We instantiate this framework with multilinear polynomials having integer coefficients, for their cryptographic utility. Validity judgements in the resulting semantics amount to a combination of range checks and polynomial constraints on the inputs, which fit tidily into the algebraic indexed relations of Zinc (Crypto '25). By compiling a suitable polynomial interactive oracle proof with the polynomial commitment scheme 'Zip' (cf. Zinc), we obtain a zk-SNARK for relations expressed in FOL. Our arithmetisation runs over the integers, obviating the need for a (costly) final step of converting the initial relations into finite field-based relations.

## CCS Concepts

• **Theory of computation** → **Interactive proof systems; Cryptographic protocols; Logic and verification**; • **Security and privacy** → **Privacy-preserving protocols; Information-theoretic techniques**; • **Mathematics of computing** → **Probabilistic algorithms**.

## Keywords

zk-SNARKs, zero knowledge, first order logic, arithmetisation  
*This authors' draft was compiled on June 18, 2026.*

## ACM Reference Format:

Murdoch J. Gabbay and Andrew Mendelsohn. 2018. zk-SNARKs for First Order Logic. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 13 pages. <https://doi.org/XXXXXXX.XXXXXXX>

## 1 Introduction

In this work we give a compositional arithmetisation of first order logic (FOL) to zk-SNARK-friendly relations for Zinc [18].

†Both authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).  
Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.  
ACM ISBN 978-1-4503-XXXX-X/2018/06  
<https://doi.org/XXXXXXX.XXXXXXX>

Our FOL arithmetisation generalises a previous univariate polynomial semantics of [15] to a generic semantics parameterised by tuples of functions ( $\mathcal{F}$  in the mathematics below) satisfying some compatibility properties. We instantiate this semantics with integer multilinear polynomials, obtaining a sound-and-complete bridge from range-checked FOL validity over finite models, to polynomial constraints compatible with Zinc.

We will now explain what FOL, zk-SNARKs, and Zinc are, and how they fit together.

*FOL.* First-order logic is a powerful mathematical logic [5, 13] which more than suffices to specify computation (including Turing-complete computation) and correctness properties thereof. Think: “Function  $f$  computes the square root” or “Abstract machine  $A$ , when given input  $x$ , returns output  $f(x)$ ” or “Transaction  $t$  is valid according to the rules of the system” or “Proposed state-change  $S$  is compliant with legal requirement  $Z$ ”, etc.

FOL is sound and complete for a simple and well-studied model-theory based on sets and functions. Its notion of truth is simple too, based on two-valued (i.e. Boolean) truth-tables.

Thus we can use FOL to simply and cleanly specify a logical or computational system *and* to state correctness properties of its behaviour.<sup>1</sup>

However, FOL has no native cryptographic content: its native truth-theoretic notions of universal and existential quantification,  $\forall x.\phi(x)$  and  $\exists x.\phi(x)$ , for example, simply mean ‘check  $\phi(x)$  for every  $x$ ’ and ‘check possible  $x$  until we find one such that  $\phi(x)$ ’.

*zk-SNARKs.* A zk-SNARK is a cryptographic tool to allow a *prover* to convince a *verifier* that the prover knows a *witness*  $w$  such that  $(x, w) \in \mathcal{R}$  for some *relation*  $\mathcal{R}$  and *instance*  $x$ , succinctly (i.e. requiring minimal work from the verifier), and in zero knowledge (i.e. the verifier does not learn information about the witness  $w$ ).

(Note that the logical content here is minimal: just an arbitrary relation  $\mathcal{R}$ . The focus here is on the cryptographic content. So even at this point, the reader can see where we are heading: we will marry the logical expressivity of FOL with the cryptographic efficiency of zk-SNARKs, using Zinc. But we anticipate...)

Most zk-SNARKs work over a fixed finite field, determined by some prime power  $q = p^r$  (e.g. [20, 16, 12]). Using finite field arithmetic allows instance-witness pairs in NP relations to be efficiently (i.e. fast prover run time) and succinctly (i.e. compact proof size) checked for validity. A common three-step modular framework has been developed for constructing zk-SNARKs:

- (1) Build a *polynomial interactive oracle proof* (PIOP).

<sup>1</sup>Actually, FOL is a family of logics: there is FOL with equality, FOL with function-symbols, FOL with reification, FOL with pattern-matching, and so on. We see no inherent barriers to adapting the FOL language of this paper with extra bells and whistles.

- (2) Build a *polynomial commitment scheme* (PCS).
- (3) Compile the PIOP+PCS to a zk-SNARK via the transform of [6, 9].

The PIOP is an information theoretic component, and the PCS provides cryptographic guarantees of the security of the construction. An overview of the construction may be found in [25, Section 9] and a study of its cryptographic properties in [22].

*The wrinkle.* A common problem in the real-world use of zk-SNARKs is that we often want to prove things that are not naturally expressed by relations  $\mathcal{R}$  defined by means of arithmetic over finite fields, much less by arithmetic over a *single* fixed finite field.

To compensate, a so-called *arithmetisation* pre-processing step is required, which transforms the  $\mathcal{R}$  that we actually want to prove, into some suitably equivalent ‘zk-SNARK friendly’  $\mathcal{R}'$ , which is over a finite field and thus more amenable to the technical requirements of zk-SNARKs.

The zk-SNARK friendly  $\mathcal{R}'$  is typically defined by a polynomial expression, which may then be rewritten to an even more zk-SNARK friendly instance of a constraint system, such as R1CS or CCS [28]. We then run the zk-SNARK on the resulting system of constraints.

For instance: we may want to prove knowledge of the result of a computation which was performed over the rationals  $\mathbb{Q}$  (which is not a finite field), or over a ring which is not a field, such as  $\mathbb{Z}/n\mathbb{Z}$  for composite  $n$ . Converting these ‘natural’ relations and constraints determined by such computations into constraints over finite fields can be computationally expensive to perform, as well as causing a significant increase in the instance size upon which a zk-SNARK is run, resulting in a costlier proof. This motivates the study of techniques to avoid or reduce the burden of arithmetisation overheads.

*Zinc.* Zinc [18] attempts to ameliorate the cost of arithmetisation. It does not make it go away, but it pushes development effort away from the programmer and onto the cryptographic back-end.

In particular, Zinc gives a PIOP for relations defined over the integers  $\mathbb{Z}$ , together with a PCS dubbed ‘Zip’, which compile to yield a zk-SNARK.

Zinc allows for succinct proofs of relations with mixed characteristic, over numerous rings, to be provided by a prover. The concept is quite simple (the devil is in the detail of making it work): for well-chosen parameters, we just sample some random primes (up to some bound) and run a finite field zk-SNARK on the relation instances modulo the sampled primes, with a low resulting soundness error. This removes the need to arithmetise such instances, beyond reducing modulo primes. We note the relations in Zinc are defined on multilinear polynomials with integer coefficients.

*Full circle back to FOL.* Previous work [15] shows how to translate FOL to polynomials with integer coefficients, such that the standard FOL truth-table-and-sets-based notion of validity may be equivalently (i.e. soundly and completely) expressed using polynomial root tests: a certain polynomial  $\langle \phi \rangle$  corresponding to the predicate  $\phi$  has roots over a certain set if and only if that predicate is true in its standard FOL semantics.

However, [15] did not explicitly connect the resulting integer polynomials to a cryptographic backend. In a happy coincidence,

just as the FOL-to-integer-polynomials translation from [15] was being introduced, the integer-multilinear-polynomial-to-zk-SNARK framework Zinc was also being introduced.

We may therefore replace the univariate FOL semantics of [15] with a multivariate semantics, instantiating the general semantics referred to in the first paragraph of this introduction with a set  $\mathcal{F}$  of multilinear interpolating polynomials of integer witness vectors. While we could choose *any* set of functions satisfying the necessary compatibility properties for our general semantics, not only functions that are interpolant polynomials, we choose multilinear polynomials with integer coefficients, long used for their cryptographic utility in building PCS [32], for compatibility with Zinc. Validity judgements in the resulting semantics (Definition 3.1) then amount to a combination of range checks and polynomial constraints on the inputs (Corollary 3.9) which fit into the *algebraic indexed relations* of [18].

Via the polynomial interactive oracle proof (PIOP) plus polynomial commitment scheme (PCS) framework [6, 9], from any compatible PIOP and Zip [18, Section 5] we obtain a zk-SNARK for statements expressed in FOL (Corollary 4.4). Since Zinc is a hash-based zk-SNARK, our construction inherits some degree of plausible post-quantum security.

*Technical map of the paper.* A FOL syntax is given in Figure 1. An integer semantics  $\langle \cdot \rangle_{\mathbb{Z}}^x$  is given in Figure 3. Note that terms evaluate to possibly negative integers, but predicates (and range checks) evaluate to natural numbers (Lemma 2.17). The integer semantics is sound and complete by Theorem 2.19.

We then consider an alternative compilation chain: a compilation  $\langle \cdot \rangle$  of FOL syntax to a novel notion of *enriched polynomial* is in Figure 2; a compilation  $mkQ_{\mathcal{F}}^x$  of enriched polynomials to multivariate polynomials is in Figure 4; and an evaluation from multivariate polynomials to integers is in Definition 3.6. Equivalence of this chain with the integer semantics is our *central technical result* in Theorem 3.8. As per Corollary 3.9, FOL validity<sup>2</sup> is equivalent to a relation determined by polynomial equalities, so that we obtain a zk-SNARK for FOL in Section 4.

*Examples.* We include two examples of computation in Section 5: exponentiation and SK combinator reduction. Both are toy examples, of course, but they still matter.

Exponentiation is a mathematical function and it is there to illustrate that we *can* define mathematical functions. It should be clear that other, more elaborate functions would just require writing other, more elaborate FOL definitions.

SK combinators may be a toy, but they are the opposite of trivial. This is a powerful and mathematically convenient Turing-complete computational model. Haskell can be compiled to SK combinators directly [4]<sup>3</sup> and only slightly more complex combinatory systems are used as practical compilation targets for industrial programming languages (a clean example is *Nock* [31]). Combinators admit a particularly compact expression in FOL, so the import of our examples is that we can arithmetise logical assertions (by definition; FOL is a logic), functions, *and* programs, covering (albeit in toy

<sup>2</sup>More precisely: a notion of validity for range-checked FOL predicates. Details in the body of the paper.

<sup>3</sup>The compilation is to SKI, but the I combinator can be expressed using S and K.

form) all the major food groups of logic and computation, so to speak.

*Related work on arithmetisation.* Other recent works have sought to address the problems of arithmetisation. Prior to Zinc, [10] aimed to build a zk-SNARK for integer relations. However, they relied on the so-called *Hidden Order Groups* assumption, which does not provide post-quantum security. There are also ‘field-agnostic’ zk-SNARKs, which can prove validity of instances of relations defined over any finite field [19, 28, 8, 36]. However, these schemes could not handle the setting of infinite fields or integral domains which are not fields. There are also schemes targeting relations over finite rings which are not fields, e.g. [17, 33, 34]. In a different direction, [21, 2] designed tools (‘Distiller’, ‘Reef’) which reduce the number of constraints required to provide a proof of the original computation. Such methods could be used concurrently with our work here. Similarly [26] is concerned with reducing the amount of ‘foreign field’ arithmetic required in proof systems. Akin to our application to SK combinators, [7, 27] design zk-SNARKs targeting computations performed in a specific programming language, in their case in C.

See [23] for an introduction to zk-SNARKs.

## 2 Basic definitions

We begin with some notation:

*Notation 2.1.* (1)  $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$  is the integers and  $\mathbb{N} = \{0, 1, 2, \dots\}$  is the non-negative integers and  $\mathbb{Q}$  is the rational numbers. (2) Given  $n \in \mathbb{N}$  write  $[n]$  for  $\{1, \dots, n\}$ . (3) Write indexed NP relations as  $\text{REL} = \{(\mathbf{i}, \mathbf{x}, \mathbf{w})\}$ . For any relation  $\text{REL}$ , let  $\mathcal{L}(\text{REL}) = \{(\mathbf{i}, \mathbf{x}) \mid \exists \mathbf{w} : (\mathbf{i}, \mathbf{x}, \mathbf{w}) \in \text{REL}\}$  denote the language corresponding to  $\text{REL}$ . (4) Write  $\mathcal{R}_\partial$  for elements of a ring  $\mathcal{R}$  of bit-length less than or equal to some bound  $\partial > 0$ . (5) Write  $\mathcal{M}_{n \times m}(\mathcal{R})$  for a matrix ring over  $\mathcal{R}$  of dimension  $n \times m$ . (6) Write  $M_{@i,j}$  for the  $j$ th element of the  $i$ th row of a matrix  $M$ . (7) We distinguish operating symbols for logical objects from mathematical operations by using  $=$ ,  $<$ , etc. rather than  $=$ ,  $<$ , etc. for operations in first order logic. (8) We use  $\llbracket \cdot \rrbracket$  to denote an oracle to a string or polynomial written within the brackets. (9) Write  $\pi_v$  for the projection taking a vector of length  $\mu \geq v$  to its  $v$ th element. Thus  $\pi_v(v_1, \dots, v_\mu) = v_v$ . (10) Given some boolean assertion  $\Phi$  (i.e. some mathematical claim that is either true or false), define  $\delta(\Phi)$  the **indicator** of  $\Phi$  such that  $\delta(\Phi) = 0$  when the assertion  $\Phi$  is true, and  $\delta(\Phi) = 1$  when  $\Phi$  is false (pun on the Dirac  $\delta$  function). (11) We may write function application as  $f(x)$  or as  $f x$ . We will use whichever notation seems clearest; meaning will always be clear.

*Definition 2.2.* (1) Given a string  $s = (s_1, s_2, \dots, s_\mu)$ , define

$$b2int(s) = \sum_{1 \leq v \leq \mu} s_v \cdot 2^{v-1}.$$

In this paper, either  $s$  will be a string of binary digits (so  $b2int(s)$  is an integer) or a string of variable symbols (so  $b2int(s)$  is a multivariate polynomial).

(2) Given  $i \in \mathbb{N}$ , write  $\mathbb{Q}_b(i)$  for the **binary decomposition** of  $i$ .

**LEMMA 2.3.** *If  $x \in \mathbb{N}$  then  $\mathbb{Q}_b(x)$  is the unique string of binary digits such that  $b2int(\mathbb{Q}_b(x)) = x$ .*

## 2.1 Multilinear Polynomials

*Definition 2.4.* A **multilinear polynomial** over a ring  $\mathcal{R}$  is a multivariate polynomial  $f(\mathbf{X}) = \sum_{i=1}^{\infty} a_i \prod_{j=1}^{\mu} X_j^{e_{ij}}$  in finitely many variables  $\mathbf{X} = (X_1, \dots, X_\mu)$  with coefficients  $a_i \in \mathcal{R}$ , such that no variable occurs in any homogeneous component with multiplicity larger than one; that is,  $0 \leq e_{ij} \leq 1$  for all  $i$  and for all  $j \in [\mu]$ . Write  $\mathcal{R}^\#[\mathbf{X}]$  for the set of multilinear polynomials in  $\mathbf{X}$  with coefficients in  $\mathcal{R}$ .

*Definition 2.5.* Given a function  $f : \{0, 1\}^\mu \rightarrow \mathcal{R}$  and a list of  $\mu$  variables  $\mathbf{X}$ , define  $\tilde{f} \in \mathcal{R}^\#[\mathbf{X}]$ , the **multilinear extension (MLE)** of  $f$ , by

$$\tilde{f}(X_1, \dots, X_\mu) = \sum_{y \in \{0, 1\}^\mu} f(y) \cdot \text{eq}(y, \mathbf{X})$$

where  $\text{eq}(y, \mathbf{X}) = \prod_{i=1}^{\mu} (y_i X_i + (1 - y_i)(1 - X_i))$

This is the unique multilinear polynomial in  $\mathcal{R}^\#[\mathbf{X}]$  which **agrees with  $f$  on the  $\mu$ -hypercube**, by which we mean that  $f(y) = \tilde{f}(y)$  for every  $y \in \{0, 1\}^\mu$  [30].

*Definition 2.6.* Given a vector  $\mathbf{v} = (v_0, \dots, v_{2^\mu-1}) \in \mathcal{R}^{2^\mu}$ , let  $f_v : \{0, 1\}^\mu \rightarrow \mathcal{R}$  take  $x$  to  $f_v(x) = v_{b2int(x)}$ , where we identify  $x$ -the-binary-string with the  $b2int(x)$ -th entry of  $\mathbf{v}$ . E.g.  $f_v(101) = v_5$ . Define the **multilinear extension** of  $\mathbf{v}$ ,  $\tilde{f}_v$ , to be the multilinear extension of  $f_v$ .

## 2.2 zk-SNARKs and Algebraic Indexed Relations

A zk-SNARK is a complete succinct non-interactive argument of knowledge satisfying knowledge soundness (up to some error  $\epsilon$ ), allowing a *prover* to convince a *verifier* of knowledge of a *witness* satisfying an *instance* of some relation  $\text{REL} = \text{REL}_{\text{gp}}$ , parameterised by *global parameters*  $\text{gp}$ .

We now explain these terms in more detail. Since we will obtain a zk-SNARK by composing a transformation from FOL to algebraic indexed relations (Definition 2.9) compatible with Zinc-PIOP [18], and then applying the transform of [6, 9] on Zinc-PIOP with the PCS Zip [18, Section 5], we borrow the following definition of interactive proofs:

*Definition 2.7 ([18, Definition 3.2]).* An **Interactive Proof (IP)** for a relation  $\text{REL}_{\text{gp}}$  with global parameters  $\text{gp}$  is a triple of algorithms  $(\text{Ind}, \text{P}, \text{V})$ , where for all  $(\mathbf{i}, \mathbf{x}, \mathbf{w}) \in \text{REL}_{\text{gp}}$ ,  $\text{Ind}$  is a deterministic algorithm taking  $(\text{gp}, \mathbf{i})$  as input and outputting verifier and prover parameters  $(\text{vp}, \text{pp}) \leftarrow \text{Ind}(\text{gp}, \mathbf{i})$ . The pair  $(\text{P}, \text{V}) = (\text{P}(\text{pp}, \mathbf{x}, \mathbf{w}), \text{V}(\text{vp}, \mathbf{x}))$  is a pair of interactive algorithms with interaction denoted  $\langle \text{P}, \text{V} \rangle$ . An IP may satisfy

(1) *Completeness with completeness error  $\epsilon_{\text{comp}}$* : for all  $(\mathbf{i}, \mathbf{x}, \mathbf{w}) \in \text{REL}_{\text{gp}}$ ,

$$\Pr[\langle \text{P}(\text{pp}, \mathbf{x}, \mathbf{w}), \text{V}(\text{vp}, \mathbf{x}) \rangle = 1 \mid (\text{vp}, \text{pp}) \leftarrow \text{Ind}(\text{gp}, \mathbf{i})] \geq 1 - \epsilon_{\text{comp}}(\text{gp}, \mathbf{i}, \mathbf{x})$$

(2) *Soundness with soundness error  $\epsilon_{\text{sound}}$* : for any unbounded adversarial prover  $\text{P}^*$  and for any  $(\mathbf{i}, \mathbf{x})$ , we have:

$$\Pr \left[ \begin{array}{c} \langle \text{P}^*(\text{pp}, \mathbf{i}, \mathbf{x}), \text{V}(\text{vp}, \mathbf{x}) \rangle = 1 \\ \wedge (\mathbf{i}, \mathbf{x}) \notin \mathcal{L}(\text{REL}_{\text{gp}}) \end{array} \mid (\text{vp}, \text{pp}) \leftarrow \text{Ind}(\text{gp}, \mathbf{i}) \right] \leq \epsilon_{\text{sound}}(\text{gp}, \mathbf{i}, \mathbf{x})$$

- (3) *Knowledge soundness with knowledge soundness error*  $\epsilon_{ks}$ : there exists a probabilistic extractor  $\text{Ext}$  such that, given oracle access to any unbounded adversarial prover  $P^*$  and any  $(i, \mathbb{x})$ , we have:

$$\Pr \left[ \begin{array}{l} \langle P^*(\text{pp}, i, \mathbb{x}), V(\text{vp}, \mathbb{x}) \rangle = 1 \\ \wedge (i, \mathbb{x}, \mathbb{w}) \notin \text{REL}_{\text{gp}} \end{array} \mid \begin{array}{l} (\text{vp}, \text{pp}) \leftarrow \text{Ind}(\text{gp}, i) \\ \mathbb{w} \leftarrow \text{Ext}_{P^*}(\text{gp}, i, \mathbb{x}) \end{array} \right] \leq \epsilon_{ks}(\text{gp}, i, \mathbb{x}, \epsilon_{P^*})$$

where

$$\epsilon_{P^*} = \epsilon_{P^*}(\text{gp}, i, \mathbb{x}) = \Pr [\langle P^*(\text{gp}, i, \mathbb{x}), V(\text{vp}, \mathbb{x}) \rangle = 1]$$

is the probability that  $P^*(\text{gp}, i, \mathbb{x})$  convinces  $V(\text{vp}, \mathbb{x})$ .

This allows us to introduce (polynomial) interactive oracle proofs:

**Definition 2.8** ([18, Definitions 3.3, 3.4]). An **interactive oracle proof (IOP)** is an interactive proof for an indexed relation  $\text{REL}_{\text{gp}} = (i, \mathbb{x}, \mathbb{w})$  in which  $i$  and  $\mathbb{x}$  may contain oracles to strings of elements from a ring  $\mathcal{R}$ .

The full strings behind the oracles from  $i$  and  $\mathbb{x}$  are contained in the prover's parameters  $\text{pp}$  and in the witness  $\mathbb{w}$ , respectively. In every protocol message, the prover  $P$  sends oracles to strings of elements from  $\mathcal{R}$ . In every round, the verifier  $V$  sends a random challenge  $\rho_i$ .

A **polynomial interactive oracle proof (PIOP)** over a ring  $\mathcal{R}$  is an IOP such that all oracles contain polynomials with coefficients in  $\mathcal{R}$  of prescribed number of variables  $\mu$  and degrees. These polynomials can be queried at any point of  $\mathcal{R}^\mu$ .

Zinc [18] develops a PIOP for multilinear relations defined on  $\mathbb{Q}$ , where inputs are restricted to have bounded bit-length  $\partial = \text{poly}(\lambda)$  in the security parameter  $\lambda$ . Here the bit-length of a rational number is computed by writing the rational number as a fraction in lowest terms, then computing the base-2 representations of the numerator and denominator (padding the smaller of the two so that they are of equal bit-size), and concatenating. This representation gives a bit-length of at most  $2(\log |a| + \log |b|) + 3$ , where an extra bit has been added to account for sign.

Zinc's relations are defined on the rationals but honest provers are expected to use integer inputs. We next formally define the relations used by Zinc. We first need some jargon: for the rest of this section, we fix an integral domain  $\mathcal{R} \subset \mathbb{Q}$ , and a tuple of global parameters  $\text{gp} = (k, m, n, \mu, \partial)$  of integers defining the magnitudes and dimensions of the objects in Definition 2.9:

**Definition 2.9.** Let  $Q$  be a set of multivariate polynomials with coefficients in a ring  $\mathcal{R}$  and let  $\mathbf{X} = (X_1, \dots, X_\mu)$  denote  $\mu$  variables. An **algebraic indexed relation (AIR)** [18, Definition 4.1] is a set  $\text{REL}_{\text{gp}, \mathcal{R}, Q}$  of triples  $(i, \mathbb{x}, \mathbb{w})$  such that:

- (1) The **index**  $i$  is an  $n$ -tuple of oracles  $\llbracket g_1 \rrbracket, \dots, \llbracket g_n \rrbracket$  to multilinear polynomials  $g_1, \dots, g_n \in \mathcal{R}_\partial^\mu[\mathbf{X}]$  for bit-length  $\partial > 0$ .
- (2) The **witness**  $\mathbb{w}$  is a  $k$ -tuple of multilinear polynomials with  $\partial$ -bounded coefficients,  $f_1, \dots, f_k \in \mathcal{R}_\partial^\mu[\mathbf{X}]$ .
- (3) The **instance**  $\mathbb{x} = (\llbracket f_1 \rrbracket, \dots, \llbracket f_k \rrbracket, \mathbf{y})$  where  $\mathbf{y} \in \mathcal{R}_\partial^m$  for some  $m \geq 0$  and the  $\llbracket f_i \rrbracket$  are oracles to the elements in  $\mathbb{w}$ .
- (4)  $Q$  is a set of multivariate polynomials with coefficients in  $\mathcal{R}$  in  $(n+k) \cdot 2^\mu + m$  variables, each of which vanishes (i.e. is equal to zero) when evaluated on the values

$$((g_1(\mathbf{x}), \dots, g_n(\mathbf{x}), f_1(\mathbf{x}), \dots, f_k(\mathbf{x}))_{\mathbf{x} \in \{0,1\}^\mu}, \mathbf{y})$$

Summing this up in symbols,  $\text{REL}_{\text{gp}, \mathcal{R}, Q}$  has the following form:

$$\text{REL}_{\text{gp}, \mathcal{R}, Q} = \left\{ (i, \mathbb{x}, \mathbb{w}) \mid \begin{array}{l} \text{gp} = (k, m, n, \mu, \partial), \\ i = (\llbracket g_1 \rrbracket, \dots, \llbracket g_n \rrbracket), \\ \mathbb{x} = (\llbracket f_1 \rrbracket, \dots, \llbracket f_k \rrbracket, \mathbf{y}) \text{ for some } \mathbf{y} \in \mathcal{R}_\partial^m, \\ \mathbb{w} = (f_1, \dots, f_k) \in (\mathcal{R}_\partial^\mu[\mathbf{X}])^k, \\ (g_1, \dots, g_n) \in (\mathcal{R}_\partial^\mu[\mathbf{X}])^n \\ Q((g_1(\mathbf{x}), \dots, g_n(\mathbf{x}), f_1(\mathbf{x}), \dots, f_k(\mathbf{x}))_{\mathbf{x} \in \{0,1\}^\mu}, \mathbf{y}) \\ = 0, \text{ for all } Q \in Q \end{array} \right\} \quad (1)$$

**Remark 2.10.** We can express CCS (an expressive constraint system) via Definition 2.9 [18, Section 4.4]. We can also express *range check/lookup* relations [28], defined by

$$\text{Look}_{\text{gp}, \mathcal{R}} = \left\{ (i, \mathbb{x}, \mathbb{w}) \mid \begin{array}{l} \text{gp} = (1, 0, 1, \log n_a + \log n_t, \partial), \\ i = (\llbracket t \rrbracket), \quad \mathbb{x} = (\llbracket a \rrbracket), \\ \mathbb{w} = (a(\mathbf{X})), \\ a(\mathbf{X}) \in \mathcal{R}_\partial^\mu[\mathbf{X}], \mathbf{X} = (X_1, \dots, X_{\log n_a}), \\ t(\mathbf{Y}) \in \mathcal{R}_\partial^\mu[\mathbf{Y}], \mathbf{Y} = (Y_1, \dots, Y_{\log n_t}), \\ \{a(\mathbf{x}) \mid \mathbf{x} \in \{0,1\}^{\log n_a}\} \\ \subseteq \{t(\mathbf{y}) \mid \mathbf{y} \in \{0,1\}^{\log n_t}\} \end{array} \right\}$$

as AIRs as follows:

Let  $a(\mathbf{X})$  and  $t(\mathbf{Y})$  be multilinear polynomials on  $\log n_a$  and  $\log n_t$  variables respectively, and let  $\mathbf{a}$  and  $\mathbf{t}$  be the vectors of evaluations of  $a(\mathbf{X})$  and  $t(\mathbf{Y})$  on the hypercubes  $\mathbf{a} = (a(\mathbf{X}))_{\{0,1\}^{\log n_a}}$  and  $\mathbf{t} = (t(\mathbf{Y}))_{\{0,1\}^{\log n_t}}$ , respectively. Define variables

$$\mathbf{W} = ((W_{\mathbf{x}})_{\mathbf{x} \in \{0,1\}^{\log n_a}}, (W_{\mathbf{y}})_{\mathbf{y} \in \{0,1\}^{\log n_t}})$$

and polynomials

$$Q_{\text{Look}} = \left\{ Q_{\mathbf{x}}(\mathbf{W}) := \prod_{\mathbf{y} \in \{0,1\}^{\log n_t}} (W_{\mathbf{x}} - W_{\mathbf{y}}) \mid \mathbf{x} \in \{0,1\}^{\log n_a} \right\}$$

Then

$$\text{REL}_{\text{gp}, \mathcal{R}, Q_{\text{Look}}} = \left\{ (i, \mathbb{x}, \mathbb{w}) \mid \begin{array}{l} \text{gp} = (1, 0, 1, \log n_a + \log n_t, \partial), \\ i = (\llbracket t \rrbracket), \quad \mathbb{x} = (\llbracket a \rrbracket), \\ a(\mathbf{X}) \in \mathcal{R}_\partial^\mu[\mathbf{X}], \\ \mathbf{X} = (X_1, \dots, X_{\log n_a}), \\ t(\mathbf{Y}) \in \mathcal{R}_\partial^\mu[\mathbf{Y}], \\ \mathbf{Y} = (Y_1, \dots, Y_{\log n_t}), \\ \mathbb{w} = (a(\mathbf{X})), \\ Q_{\mathbf{x}}(\mathbf{a}, \mathbf{t}) = 0, \\ \text{for all } \mathbf{x} \in \{0,1\}^{\log n_a} \end{array} \right\} \quad (2)$$

is equivalent to the (standard) lookup relation  $\text{Look}_{\text{gp}}$  [18, Section 4.4].

Call an index-instance-witness triple for an algebraic indexed relation  $\text{REL}$  **well-formed** when it is of the form specified in Definition 2.9. In particular, we assume throughout that malicious provers and adversaries use well-formed instances of AIRs, and we assume that  $\mathcal{R} = \mathbb{Z}$ . We thus remove the subscripted ring in our relation definitions and write  $\text{REL}_{\text{gp}, Q}$  for AIRs.

We refer the reader to [18, Definition 3.5] for the definition of a PCS. A multilinear PCS is **succinct** if it outputs commitments which are sublinear in  $2^\mu$ .

|        |                                                                                                                                                                     |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Term   | $t ::= q \mid t + t \mid t * t \mid \text{len}(C) \mid \text{reify}(\phi) \mid$<br>$X \mid C_i(X) \mid C_i C_j(X)$<br>$(i, j \in [\text{ar}(C)], q \in \mathbb{Z})$ |
| Pred   | $\phi, \psi ::= t = t \mid \phi \wedge \phi \mid \phi \vee \phi$                                                                                                    |
| RCheck | $R ::= \emptyset \mid R, t < C_i \mid R, C_i < t$<br>$(i \in [\text{ar}(C)])$                                                                                       |
| EP     | $P ::= q \mid P + P \mid P * P \mid \text{len}(C) \mid X \mid C_i(X) \mid C_i C_j(X)$<br>$(i, j \in [\text{ar}(C)], q \in \mathbb{Z})$                              |

**Figure 1: Terms, predicates, range checks, and enriched polynomials (Definition 2.12(1))**

### 2.3 First-order logic

We set up the syntax and denotation of a simple first-order logic:

*Notation 2.11.* Here and for the rest of the paper we fix the following syntactic data:

- (1) A single **matrix variable symbol**  $C \in \text{MVS}$  with a fixed but arbitrary arity  $\text{ar}(C) \geq 1$ . (MVS might contain other symbols, but we will only need one for this paper.)
- (2) A set **BVar** of **quad-indexed variable symbols**  $B_{i,v}^{j,x} \in \text{BVar}$ , where  $i, j, x, v \in \mathbb{N}$ .
- (3) A single **index variable symbol**  $X$ .

These are all just formal symbols. We assume they are all distinct.

A sound and complete polynomial semantics for first-order logic (FOL) from [15] used univariate polynomials and their properties to precisely (i.e. soundly and completely) capture the logical content of FOL predicates.<sup>4</sup> We now construct a simplified yet expressive FOL syntax (Figure 1) and a version of the semantics in [15] (Figure 3) which are tailored to our needs in this paper.

*Definition 2.12.* (1) Define syntaxes of **terms**, **predicates**, and **enriched polynomials** in Figure 1.  
(2) We may write  $\top$  as sugar for  $(0 = 0)$ , and  $\perp$  for  $(0 = 1)$ .  
(3) Define a mapping  $\langle \cdot \rangle$  taking terms and predicates to enriched polynomials in Figure 2.

*Remark 2.13 (Some comments on Definition 2.12).* Pred is a simple predicate language of conjunctions and disjunctions over equalities. We can express true as  $0 = 0$  and false as  $0 = 1$ . A quantification over the index variable symbol  $X$  is implicit in the syntax, as will become explicit later in Definition 3.1(3&4) and Lemma 3.2.

Pred only has one index variable symbol  $X$ , and one polynomial function symbol  $C$ . In general we might want more (if only for convenience), but even in its current form Pred is actually very expressive.<sup>5</sup> So Pred may be compact, but it is not a toy: in particular, it can express the full list of examples from [15].

Enriched polynomials are just a subsyntax of terms: this is deliberate, and we will exploit it below. We do not use examples that use reify in this paper *but* we retain it just to make clearer that the syntax here is fully as expressive as that of [15] (albeit more concisely presented).

<sup>4</sup>A cryptographically oriented version is in [14].

<sup>5</sup>Logicians please note that reify is extremely powerful. This syntax is much more expressive than it looks.

**LEMMA 2.14.** *If  $t \in \text{Term}$  and  $\phi \in \text{Pred}$  then  $\langle t \rangle$  and  $\langle \phi \rangle$  are enriched polynomials.*

**PROOF.** By a routine induction.  $\square$

*Remark 2.15.* Why are  $\langle \cdot \rangle$  and enriched polynomials interesting?

- (1) We can map  $t \in \text{Term}$  and  $\phi \in \text{Pred}$  (Figure 1) to enriched polynomials as per Lemma 2.14.
- (2) We can map enriched polynomials to *actual* multivariate polynomials, as per  $mkQ_{\mathcal{F}}^x$  in Figure 4.
- (3) We can evaluate the variables in the resulting polynomials generated by  $mkQ_{\mathcal{F}}^x$  using  $\beta_{\mathcal{F}}$  (Definition 3.6), such that (by Theorem 3.8) the composition  $\beta_{\mathcal{F}} \circ mkQ_{\mathcal{F}}^x \circ \langle \cdot \rangle$  coincides (by Theorem 2.19) with the sound and complete integer semantics for FOL  $\langle \cdot \rangle_{\zeta}^x$  from Figure 3.
- (4) We can use the above machinery to generate polynomial relations compatible with Zinc.

### 2.4 Integer semantics for FOL

In this section we define  $\langle \cdot \rangle_{\zeta}^x$  (Figure 3), describing a sound and complete integer semantics for FOL.

*Definition 2.16 (FOL semantics).* (1) We define an **interpretation**  $\zeta : \text{MVS} \rightarrow \mathcal{M}_{\text{ar}(C) \times \ell_{\zeta}(C)}(\mathbb{N})$  to map  $C \in \text{MVS}$  to an  $\text{ar}(C) \times \ell_{\zeta}(C)$  non-negative integer matrix, where  $\ell_{\zeta}(C) \in \mathbb{N}_{\geq 1}$  depends on  $\zeta$ .<sup>6</sup>  
(2) Suppose  $t \in \text{Term}$  and  $\phi \in \text{Pred}$  and  $R \in \text{RCheck}$  and  $x \in [\ell_{\zeta}(C)]$  and  $\zeta$  is an interpretation. Define a **semantics**  $\langle t \rangle_{\zeta}^x \in \mathbb{Z}$ ,  $\langle \phi \rangle_{\zeta}^x, \langle R \rangle_{\zeta} \in \mathbb{N}$  as per Figure 3.

Definition 2.16 might seem odd: FOL already has a well-known Boolean truth-valued semantics, so: a) why give it an integer semantics, and b) what does this integer semantics even mean? We answer as follows: we give FOL an integer semantics because it will help us arithmetise FOL in Section 3. As to what the semantics means, we can sum this up using two slogans:

- (1) *Slogan 1.* Zero = true; non-zero = false. We treat  $\mathbb{N}$  as a domain of truth-values in which zero is the single designated ‘true’ truth-value, and non-zero values are flavours of ‘false’.
- (2) *Slogan 2.* Equality = square of difference; conjunction = sum; disjunction = product. This just reads off the relevant clauses in Figure 3.

These slogans are mathematically justified by Lemma 2.17 and Theorem 2.19.

**LEMMA 2.17 (NON-NEGATIVITY).** *Suppose  $\phi \in \text{Pred}$  and  $\zeta$  is an interpretation. Then  $\langle \phi \rangle_{\zeta}^x \geq 0$ .*

**PROOF.** We consider  $\langle \phi \rangle_{\zeta}^x$  in Figure 3:  $\langle t = t' \rangle_{\zeta}^x$  is a square and squares are non-negative (even if  $\langle t \rangle_{\zeta}^x$  or  $\langle t' \rangle_{\zeta}^x$  are negative);  $\langle \phi \wedge \phi' \rangle_{\zeta}^x$

<sup>6</sup>Some words on the design here: Letting  $\zeta(C)$  be a matrix of *non-negative* numbers is convenient because it slightly simplifies  $b2int$  and  $\mathbf{0}_b$  in Definition 2.2 (no need to worry about binary representations of negative numbers). We can still get negative numbers in terms  $t$  using addition and multiplication by  $q \in \mathbb{Z}$ .

Perhaps we should call this a *numerical* semantics for FOL, but in the context of cryptography that might be misread as a semantics in finite fields. If this bothers the reader, they can substitute ‘numerical’ or ‘numbers-based’ every time we write *integer*, and no harm will come of it. We will always specify precise types, so meaning will be clear.

$$\begin{aligned}
\langle q \rangle &= q \quad (q \in \mathbb{Z}) & \langle t + t' \rangle &= \langle t \rangle + \langle t' \rangle & \langle t * t' \rangle &= \langle t \rangle * \langle t' \rangle & \langle \text{len}(C) \rangle &= \text{len}(C) \\
\langle X \rangle &= X & \langle C_i(X) \rangle &= C_i(X) & \langle C_i C_j(X) \rangle &= C_i C_j(X) & \langle \text{reify}(\phi) \rangle &= \langle \phi \rangle \\
\langle t = t' \rangle &= (\langle t \rangle - \langle t' \rangle)^2 & \langle \phi \wedge \phi' \rangle &= \langle \phi \rangle + \langle \phi' \rangle & \langle \phi \vee \phi' \rangle &= \langle \phi \rangle * \langle \phi' \rangle
\end{aligned}$$

Figure 2: From Term and Pred to enriched polynomials EP (Definition 2.12(3))

$$\begin{aligned}
\langle q \rangle_\zeta^x &= q & \langle t + t' \rangle_\zeta^x &= \langle t \rangle_\zeta^x + \langle t' \rangle_\zeta^x & \langle t * t' \rangle_\zeta^x &= \langle t \rangle_\zeta^x * \langle t' \rangle_\zeta^x & \langle \text{len}(C) \rangle_\zeta^x &= \text{len}(\zeta(C)) \\
\langle X \rangle_\zeta^x &= x & \langle C_i(X) \rangle_\zeta^x &= \zeta(C) @_{i,x} & \langle C_i C_j(X) \rangle_\zeta^x &= \zeta(C) @_{i,\zeta(C) @_{j,x}} & \langle \text{reify}(\phi) \rangle_\zeta^x &= \langle \phi \rangle_\zeta^x \\
\langle t = t' \rangle_\zeta^x &= (\langle t \rangle_\zeta^x - \langle t' \rangle_\zeta^x)^2 & \langle \phi \wedge \phi' \rangle_\zeta^x &= \langle \phi \rangle_\zeta^x + \langle \phi' \rangle_\zeta^x & \langle \phi \vee \phi' \rangle_\zeta^x &= \langle \phi \rangle_\zeta^x * \langle \phi' \rangle_\zeta^x \\
\langle \emptyset \rangle_\zeta &= 0 & \langle R, C_i < t \rangle_\zeta &= \langle R \rangle_\zeta + \delta(\forall x \in [\text{len}(\zeta(C))]. \langle t \rangle_\zeta^x > \zeta(C) @_{i,x}) \\
& & \langle R, t < C_i \rangle_\zeta &= \langle R \rangle_\zeta + \delta(\forall x \in [\text{len}(\zeta(C))]. \langle t \rangle_\zeta^x < \zeta(C) @_{i,x})
\end{aligned}$$

Above:  $\delta$  is the indicator function from Notation 2.1(10), and  $@_{i,x}$  is from Notation 2.1(6).

Figure 3: Integer semantics for FOL terms, predicates, and range checks (Definition 2.16)

is a sum and sums of non-negatives are non-negative;  $\langle \phi \vee \phi' \rangle_\zeta^x$  is a product and products of non-negatives are non-negative.  $\square$

**Definition 2.18 (Validity).** Suppose  $\phi \in \text{Pred}$  and  $R \in \text{RCheck}$  and  $\zeta$  is an interpretation on  $C$  and  $x \in [\text{len}(\zeta(C))]$ . Then write  $x \models_\zeta \phi$  when  $\langle \phi \rangle_\zeta^x = 0$ , and write  $\models_\zeta R$  when  $\langle R \rangle_\zeta = 0$ .

**THEOREM 2.19 (SOUNDNESS AND COMPLETENESS).** Suppose  $\phi, \phi' \in \text{Pred}$ , and  $t, t' \in \text{Term}$ , and  $R \in \text{RCheck}$ . Suppose  $\zeta$  is an interpretation, for brevity write  $\ell = \text{len}(\zeta(C))$ , and suppose  $x \in [\ell]$ . Then:

- (1)  $x \models_\zeta t = t'$  if and only if  $\langle t \rangle_\zeta^x = \langle t' \rangle_\zeta^x$ .
- (2)  $x \models_\zeta \top$  and  $x \not\models_\zeta \perp$  (by Definition 2.12(2)  $\top$  is sugar for  $0 = 0$  and  $\perp$  is sugar for  $0 = 1$ ).
- (3)  $x \models_\zeta \phi \wedge \phi'$  if and only if  $x \models_\zeta \phi \wedge x \models_\zeta \phi'$ .
- (4)  $x \models_\zeta \phi \vee \phi'$  if and only if  $x \models_\zeta \phi \vee x \models_\zeta \phi'$ .
- (5)  $\models_\zeta \emptyset$ .
- (6)  $\models_\zeta R, t < C_i$  if and only if  $\models_\zeta R \wedge \forall x \in [\ell]. \langle t \rangle_\zeta^x < \zeta(C) @_{i,x}$ .
- (7)  $\models_\zeta R, C_i < t$  if and only if  $\models_\zeta R \wedge \forall x \in [\ell]. \zeta(C) @_{i,x} < \langle t \rangle_\zeta^x$ .

**PROOF.** We reason as follows ([15, Theorem 2.4.4] has an analogous proof):

- (1) It is a fact that  $0 = 0$  and  $1 \neq 0$ .
- (2) It is a fact that  $x = x'$  if and only if  $(x - x')^2 = 0$ .
- (3) By non-negativity (Lemma 2.17).
- (4) The product of two non-negative numbers is non-zero if and only if they both are.
- (5) It is a fact that  $0 = 0$ .
- (6) By construction: range checks compile to 0 and 1 (depending on values of the indicator function  $\delta$ ). Their sum is zero if and only if all summands are zero.
- (7) As for the previous item.  $\square$

### 3 Arithmetising FOL

We now show how to arithmetise the FOL syntax from Figure 1, i.e. to map FOL to polynomials that match up, in a sense made formal by Theorem 3.8, with the integer semantics of Theorem 2.19.

**Definition 3.1.** (1) Say  $\phi \in \text{Pred}$  **uses**  $C_j$  **as a pointer** when  $\phi$  contains a subterm of the form  $C_i C_j(X)$ .

- (2) Say  $(\phi, R) \in \text{Pred} \times \text{RCheck}$  is **range-checked** when for every  $j$  such that  $\phi$  uses  $C_j$  as a pointer,  $R$  contains conditions of the form  $0 < C_j$  and  $C_j < \text{len}(C) + 1$ .
- (3) A **judgement** is a 3-tuple, written  $C \models_\zeta \phi; R$ , of: an interpretation  $\zeta$ ; a predicate  $\phi \in \text{Pred}$ ; and a range check  $R \in \text{RCheck}$ ; such that  $(\phi, R)$  is range-checked.
- (4) Call a judgement  $C \models_\zeta \phi; R$  **valid** when:  $\langle \phi \rangle_\zeta^x = 0$  for every  $x \in [\text{len}(\zeta(C))]$ , and  $\langle R \rangle_\zeta = 0$ .
- (5) We may write  $C \models_\zeta \phi; \emptyset$  just as  $C \models_\zeta \phi$ , and we may write  $C \models_\zeta \top; R$  just as  $C \models_\zeta R$ .

**LEMMA 3.2.** Suppose  $\phi \in \text{Pred}$ ,  $R \in \text{RCheck}$ , and  $\zeta$  is an interpretation. Write  $\ell = \text{len}(\zeta(C))$ . Then:

- (1)  $C \models_\zeta \phi$  when  $\langle \phi \rangle_\zeta^x = 0$  (equivalently using Definition 2.18:  $x \models_\zeta \phi$ ) for every  $x \in [\ell]$ .  
Note this gives  $C \models_\zeta \phi$  the flavour of a universal quantification of  $\phi$  for  $X$  ranging over  $x \in [\ell]$ .
- (2)  $C \models_\zeta R$  when  $\langle R \rangle_\zeta = 0$  (equivalently:  $\models_\zeta R$ ).  
We unpack what this means using Theorem 2.19(6&7): for every  $x \in [\ell]$ , we have  $\langle t \rangle_\zeta^x < \zeta(C) @_{i,x}$  for every  $t < C_i$  appearing in  $R$ , and  $\langle t \rangle_\zeta^x > \zeta(C) @_{i,x}$  for every  $C_i < t$  appearing in  $R$ .

**PROOF.** Direct from Definition 3.1.  $\square$

**Remark 3.3.** As per Lemma 3.2,  $C \models_\zeta \phi; R$  being valid has the flavour of a universal quantification that  $\phi$  and  $R$  are valid for every  $x \in [\ell]$ . This gives our syntax from Figure 1 some extra expressivity, in that the index variable  $X$  is universally quantified over  $[\ell]$ .

**Remark 3.4.** We continue Remark 2.15: predicates  $\phi \in \text{Pred}$  (Figure 1) can express equality, conjunction, and disjunction on terms that include  $C_i(X)$  and  $C_i C_j(X)$ . Range checks  $R \in \text{RCheck}$  can express range checks on the  $C_i$ .

The integer semantics from Figure 3 maps this syntax to integers, and Theorem 2.19 shows that this fits together in the sense that with these definitions, validity of predicates and range checks behaves in the way that the symbols would lead us to expect. For example:  $x \models_\zeta \phi \wedge \phi'$  is indeed valid if and only if  $x \models_\zeta \phi$  is valid and  $x \models_\zeta \phi'$  is valid; and  $t < C_i$  is indeed valid when  $\langle t \rangle_\zeta^x$  is a lower bound for the  $i$ th row of  $\zeta(C)$ .

As per Remark 2.15, we will now connect our integer semantics to a multivariate polynomial semantics which we build using the enriched polynomials from Figure 2.

We now define  $mkQ$  (read ‘make  $Q$ ’) taking enriched polynomials to multivariate polynomials. Intuitively,  $mkQ_{\mathcal{F}}^x$  is what computes the polynomials  $Q \in \mathcal{Q}$  used in equation (1). We pad binary expansions of the elements of the images of the  $f_i \in \mathcal{F}$  so that all binary expansions have the same length, equal to the largest bit-length obtaining over all the ranges of the  $f_i$ . Write  $maxbl$  for the *maximum bit-length* of the elements in the images of the  $f_i$ .

**Definition 3.5.** Suppose  $maxbl, \ell > 0$ ,  $x \in [\ell]$ , and  $\mathcal{F} = (f_i : [\ell] \rightarrow \mathbb{N} \mid i \in [ar(C)])$ . Define  $mkQ_{\mathcal{F}}^x$  mapping enriched polynomials (Definition 2.12(1)) to multivariate polynomials

$$mkQ_{\mathcal{F}}^x : \mathcal{EP} \rightarrow \mathbb{Z}[B_{1,1}^{0,1}, \dots, B_{1,maxbl}^{0,1}, \dots, B_{ar(C),1}^{ar(C),\ell}, \dots, B_{ar(C),maxbl}^{ar(C),\ell}]$$

as per Figure 4.

Next we define an evaluation map  $\beta_{\mathcal{F}}$  as a composition of the  $\pi_v$ ,  $\mathbb{Q}_b$ , and the  $f_i$ :

**Definition 3.6.** Suppose  $maxbl, \ell > 0$  and  $\mathcal{F} = (f_i : [\ell] \rightarrow \mathbb{N} \mid i \in [ar(C)])$ .<sup>7</sup> Then:

- (1) For each  $i \in [ar(C)]$  and  $j \in \{0\} \cup [ar(C)]$  and  $x \in [\ell]$  and  $v \in [maxbl]$ , we define  $\beta_{\mathcal{F}}(B_{i,v}^{0,x})$  and  $\beta_{\mathcal{F}}(B_{i,v}^{j,x})$  by

$$\begin{aligned} \beta_{\mathcal{F}}(B_{i,v}^{0,x}) &= \pi_v \mathbb{Q}_b f_i(x) \quad \text{and} \\ \beta_{\mathcal{F}}(B_{i,v}^{j,x}) &= \begin{cases} \pi_v \mathbb{Q}_b f_i(f_j(x)) & \text{if } f_j(x) \in [\ell] \\ 0 & \text{if } f_j(x) \notin [\ell] \end{cases} \end{aligned}$$

Thus in words:  $\beta_{\mathcal{F}}$  maps  $B_{i,v}^{0,x}$  to the  $v$ th bit of the binary expansion of  $f_i(x)$ ; and  $\beta_{\mathcal{F}}$  maps  $B_{i,v}^{j,x}$  to the  $v$ th bit of the binary expansion of  $f_i(f_j(x))$  (if this is in-range).

- (2) We extend  $\beta_{\mathcal{F}}$  to an **evaluation map** on multivariate polynomials, in the natural way by instantiating variable symbols. For example:  $\beta_{\mathcal{F}}(0 + B_{1,3}^{0,2} * B_{4,6}^{0,5}) = 0 + (\pi_3 \mathbb{Q}_b f_1(2)) * (\pi_6 \mathbb{Q}_b f_4(5))$ .<sup>8</sup>

**Definition 3.7.** Suppose  $\mathcal{F} = (f_i : [\ell] \rightarrow \mathbb{N} \mid i \in [ar(C)])$  and  $t \in \text{Term}$  and  $\phi \in \text{Pred}$ . Call  $\mathcal{F}$  **in-range** for  $t$  / for  $\phi$  when for every  $j$  that  $t$  /  $\phi$  uses as a pointer, we have  $\forall x \in [\ell]. 1 \leq f_j(x) \leq \ell$ .

A central technical result is that  $\beta_{\mathcal{F}} \circ mkQ_{\mathcal{F}}^x \circ \langle \cdot \rangle$  equals  $\langle \cdot \rangle_{\zeta}^x$ :

**THEOREM 3.8.** Suppose  $t \in \text{Term}$  and  $\phi \in \text{Pred}$  and  $\zeta$  is an interpretation. Write  $\ell = \text{len}(\zeta(C))$  and suppose  $x \in [\ell]$ . Define  $\mathcal{F} = (f_i = (x \in [\ell] \mapsto \zeta(C)_{@i,x}) \mid i \in [ar(C)])$ . If  $\mathcal{F}$  is in-range for  $t$  and  $\phi$  (Definition 3.7), then:

$$\beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x(t)) = \langle t \rangle_{\zeta}^x \quad \text{and} \quad \beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x(\phi)) = \langle \phi \rangle_{\zeta}^x$$

<sup>7</sup>In the case of a cryptographic application to arithmetise FOL relations to AIRs, a verifier  $V$  will not have direct access to the  $f_i$ , instead having oracle access to them via a polynomial commitment scheme.

<sup>8</sup>Unpicking the symbol salad, this is zero plus the AND of the third bit of the binary expansion of  $f_1(2)$  with the sixth bit of the binary expansion of  $f_4(5)$ . (Why? It's an example.)

**PROOF.** By a routine induction on syntax. The interesting cases are for  $X$ ,  $C_i(X)$ , and  $C_i C_j(X)$ :

$$\begin{aligned} \beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x \langle X \rangle) &= \beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x X) && \text{Figure 2} \\ &= \beta_{\mathcal{F}}(x) && \text{Figure 4} \\ &= x && \text{Def. 3.6(2)} \\ &= \langle X \rangle_{\zeta}^x && \text{Figure 3} \end{aligned}$$

$$\begin{aligned} \beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x \langle C_i(X) \rangle) &= \beta_{\mathcal{F}}(b2int(B_{i,1}^{0,x}, \dots, B_{i,maxbl}^{0,x})) && \text{Figures 2 \& 4} \\ &= b2int(\mathbb{Q}_b f_i(x)) && \text{Def. 3.6} \\ &= f_i(x) && \text{Lemma 2.3} \\ &= \zeta(C)_{@i,x} && \text{Def. of } \mathcal{F}, x \in [\ell] \\ &= \langle C_i(x) \rangle_{\zeta}^x && \text{Figure 3} \end{aligned}$$

$$\begin{aligned} \beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x \langle C_i C_j(X) \rangle) &= \beta_{\mathcal{F}}(b2int(B_{i,1}^{j,x}, \dots, B_{i,maxbl}^{j,x})) && \text{Figures 2 \& 4} \\ &= f_i(f_j(x)) && \text{Def. 3.6 \& Lemma 2.3} \\ &= \zeta(C)_{@i,\zeta(C)_{@j,x}} && \text{Def. of } \mathcal{F}, f_j(x) \in [\ell] \\ &= \langle C_i C_j(X) \rangle_{\zeta}^x && \text{Figure 3} \end{aligned}$$

□

**COROLLARY 3.9.** Suppose  $\zeta$  is an interpretation such that  $\zeta(C) \in \mathcal{M}_{ar(C) \times \ell}(\mathbb{N})$ . Suppose  $C \models \phi$ ;  $R$  is a judgement (so that  $\phi \in \text{Pred}$  and  $R \in \text{RCheck}$  and  $(\phi, R)$  is properly range-checked). Define  $\mathcal{F} = (f_i = (x \in [\ell] \mapsto \zeta(C)_{@i,x}) \mid i \in [ar(C)])$ . Then:

$C \models_{\zeta} \phi$ ;  $R$  if and only if

$$\begin{aligned} \forall x \in [\ell]. \beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x \langle \phi \rangle) &= 0 \quad \wedge \\ \forall x \in [\ell]. \beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x \langle t \rangle) &< \zeta(C)_{@i,x} \text{ for every } t < C_i \text{ in } R \quad \wedge \\ \forall x \in [\ell]. \beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x \langle t \rangle) &> \zeta(C)_{@i,x} \text{ for every } C_i < t \text{ in } R \end{aligned}$$

**PROOF.** Routine from Theorem 3.8 and Lemma 3.2. The theorem requires  $\mathcal{F}$  to be in-range (Definition 3.7); this follows from our assumption that  $(\phi, R)$  is range-checked (Definition 3.1(2)). □

## 4 A zk-SNARK for Relations in First Order Logic

Below, we describe in Lemma 4.2 how to transform instances of relations defined in first order logic to instances of relations defined using multivariate polynomials, using Corollary 3.9. These latter relations are suitable inputs to Zinc-PIOP. We do not require auxiliary data in the index  $\mathbf{i}$ , so we only consider instance-witness pairs  $(\mathbf{x}, \mathbf{w})$  below. We first define relations for predicates in  $\text{Pred}$  and range checks in  $\text{RCheck}$ :

**Definition 4.1.** Suppose  $\phi \in \text{Pred}$  (the predicate language from Figure 1), and  $R \in \text{RCheck}$ , and  $\zeta$  is an interpretation. Let  $\ell = \text{len}(\zeta(C))$ . Define two relations:

$$\begin{aligned} \text{REL}_{\text{gp}}^{\text{FOL}} &= \left\{ ((\phi, R), \zeta) \mid \begin{array}{l} \zeta(C) \in \mathcal{M}_{ar(C) \times \ell}(\mathbb{N}) \\ C \models_{\zeta} \phi; R \end{array} \right\} \quad \text{and} \\ \text{REL}_{\text{gp}}^{\text{EP}} &= \left\{ ((\langle \phi \rangle, R), \zeta(C)) \mid \begin{array}{l} \zeta(C) \in \mathcal{M}_{ar(C) \times \ell}(\mathbb{N}) \\ C \models_{\zeta} \phi; R \end{array} \right\} \end{aligned}$$

The relation  $\text{REL}_{\text{gp}}^{\text{EP}}$  is an intermediate relation obtained by turning instances of  $\text{REL}_{\text{gp}}^{\text{FOL}}$  into enriched polynomials, via Figure 2. By converting  $\phi$  to  $\langle \phi \rangle$ , we obtain an enriched polynomial instance

$$\begin{aligned} mkQ_{\mathcal{F}}^x(q) &= q & mkQ_{\mathcal{F}}^x(P + P') &= mkQ_{\mathcal{F}}^x(P) + mkQ_{\mathcal{F}}^x(P') & mkQ_{\mathcal{F}}^x(P * P') &= mkQ_{\mathcal{F}}^x(P) * mkQ_{\mathcal{F}}^x(P') & mkQ_{\mathcal{F}}^x(\text{len}(C)) &= \ell \\ mkQ_{\mathcal{F}}^x(X) &= x & mkQ_{\mathcal{F}}^x(C_i(X)) &= b2int(B_{i,1}^{0,x}, \dots, B_{i,maxbl}^{0,x}) & mkQ_{\mathcal{F}}^x(C_i C_j(X)) &= b2int(B_{i,1}^{j,x}, \dots, B_{i,maxbl}^{j,x}) \end{aligned}$$

Above,  $i \in [ar(C)]$ ,  $j \in \{0\} \cup [ar(C)]$ , and  $maxbl, \ell \in \mathbb{N}_{\geq 1}$  and  $x \in [\ell]$  and  $\mathcal{F} = (f_i : [\ell] \rightarrow \mathbb{N} \mid i \in [ar(C)])$  is an  $ar(C)$ -tuple of functions. Note that  $j \in \{0\} \cup [ar(C)]$  yet  $C_0(X)$  is not defined - this will not trouble us because a FOL validity predicate will never transform to an enriched polynomial supported on  $C_0(X)$ . Thus the clauses for  $C_i(X)$  and  $C_i C_j(X)$  have disjoint images.

**Figure 4: From enriched polynomials EP to multivariate polynomials MVP (Definition 3.5)**

preserving the logical content of  $\phi$ , with a witness  $w = \varsigma$  being knowledge of  $\varsigma(C) \in \mathcal{M}_{ar(C) \times \ell}(\mathbb{N})$  satisfying  $\phi$ .

We now use  $mkQ_{\mathcal{F}}^x$  (Figure 4) to map enriched polynomials to multivariate polynomials, setting  $\mathcal{F} = (\tilde{f}_{\varsigma(C)}^i \mid i \in [ar(C)])$ , i.e.  $\mathcal{F}$  comprises the multilinear extensions of the rows of  $\varsigma(C)$ . Note that, before padding, the  $\tilde{f}_{\varsigma(C)}^i$  are multilinear polynomials in  $\mu = \lceil \log_2 \ell \rceil$  variables. However, for pointer correctness, we pad so that  $\mu = \max(\lceil \log_2 \ell \rceil, maxbl)$ .<sup>9</sup>

By Corollary 3.9, with  $\mathcal{F} = (\tilde{f}_{\varsigma(C)}^i \mid i \in [ar(C)])$  and defining  $\tilde{f}_{\varsigma(C)}^i(\tilde{f}_{\varsigma(C)}^j(x)) := \tilde{f}_{\varsigma(C)}^i(\mathbb{0}_b \tilde{f}_{\varsigma(C)}^j(x))$ , we have that  $\text{REL}_{\text{gp}}^{\text{EP}}$  of Definition 4.1 is equivalent to

$$\text{REL}_{\text{gp}, \mathcal{F}}^{\text{MV}} = \left\{ \begin{array}{l} (((mkQ_{\mathcal{F}}^x(\phi))_{x \in [\ell]}, R), (\tilde{f}_{\varsigma(C)}^i)_{i \in [ar(C)]}) \\ \left| \begin{array}{l} \varsigma(C) \in \mathcal{M}_{ar(C) \times \ell}(\mathbb{N}), C \models_{\varsigma} R, \\ \forall x \in [\ell] (\beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x(\phi)) = 0) \end{array} \right. \end{array} \right\}$$

By choosing  $\mathcal{F} = \{\tilde{f}_{\varsigma(C)}^i \mid i \in [ar(C)]\}$ , we find that the condition that the image of  $\beta_{\mathcal{F}}$  on the multivariate polynomial  $mkQ_{\mathcal{F}}^x(\phi)$  satisfies  $\beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x(\phi)) = 0$  is equivalent to  $mkQ_{\mathcal{F}}^x(\phi)$  evaluating to zero at some subset of

$$\begin{aligned} &(\mathbb{0}_b(\varsigma(C)_{@1,1}), \dots, \mathbb{0}_b(\varsigma(C)_{@ar(C), \ell})) \\ &= (\mathbb{0}_b \tilde{f}_{\varsigma(C)}^1(\mathbb{0}_b 1), \dots, \mathbb{0}_b \tilde{f}_{\varsigma(C)}^{ar(C)}(\mathbb{0}_b \ell)) \end{aligned}$$

(recall from Notation 2.1(6) that  $\varsigma(C)_{@i,j}$  is the  $i, j$ th entry of  $\varsigma(C)$ ) as long as the range checks in  $R$  guarantee that any pointer  $C_i(X)$  in  $\phi$  corresponds to a row  $\varsigma(C)_i$  with entries bounded by  $\text{len}(\varsigma(C))$ .

We now prove that  $\text{REL}_{\text{gp}, \mathcal{F}}^{\text{MV}}$  provides algebraic indexed relations:

**LEMMA 4.2.** *Let  $\text{gp}_0, \text{gp}_1$  denote global parameters, and fix  $\partial, \mu > 0$ . Let  $\phi$  be a predicate defined over  $\mathbb{Z}$  in a matrix variable symbol  $C$ . Let  $\varsigma(C) \in \mathcal{M}_{ar(C) \times \ell}(\mathbb{N})$  be an interpretation, and  $R$  be a set of range check conditions on  $\varsigma(C)$ . Set  $\mathcal{F} = \{\tilde{f}_{\varsigma(C)}^i \in \mathbb{Z}_{\partial}^{\mu}[\mathbf{X}] \mid i \in [ar(C)]\}$ . Then the data of an instance of  $\text{REL}_{\text{gp}_0}^{\text{EP}}$  can be expressed as a pair  $(\mathbb{x}, w)$  of  $\text{REL}_{\text{gp}_1, Q}$  for some  $\text{gp}_1$  and some  $Q$ , such that  $(\mathbb{x}, w)$  is well-formed if and only if  $C \models_{\varsigma} \phi$ ;  $R$  is valid.*

**PROOF.** Let

$$(((mkQ_{\mathcal{F}}^x(\phi))_{x \in [\text{len}(\varsigma(C))]}, R), (\tilde{f}_{\varsigma(C)}^i)_{i \in [ar(C)]}) \in \text{REL}_{\text{gp}, \mathcal{F}}^{\text{MV}}$$

be obtained from some instance  $((\phi, R), \varsigma) \in \text{REL}_{\text{gp}_0}^{\text{EP}}$ . We begin by considering  $(mkQ_{\mathcal{F}}^x(\phi))_{x \in [\text{len}(\varsigma(C))]}$ : we define an AIR,  $\text{REL}_{\text{gp}', Q'}$ ,

corresponding to this data. Set the instance-witness pair in  $\text{REL}_{\text{gp}', Q'}$  to be

$$(\mathbb{x}, w) = ((\llbracket \tilde{f}_{\varsigma(C)}^1 \rrbracket, \dots, \llbracket \tilde{f}_{\varsigma(C)}^{ar(C)} \rrbracket), (\tilde{f}_{\varsigma(C)}^1, \dots, \tilde{f}_{\varsigma(C)}^{ar(C)}))$$

where  $\llbracket \cdot \rrbracket$  denotes an oracle to the argument,<sup>10</sup> and

$$Q' = (mkQ_{\mathcal{F}}^x(\phi))_{x \in [\text{len}(\varsigma(C))]}$$

We claim this data represents an AIR for some  $\text{gp}'$ : note that for each  $x \in [\text{len}(\varsigma(C))]$ ,  $mkQ_{\mathcal{F}}^x(\phi)$  is a multivariate polynomial determined by  $\phi$ , evaluating to zero on a subset of the binary decompositions

$$(\mathbb{0}_b \tilde{f}_{\varsigma(C)}^1(\mathbb{0}_b 1), \dots, \mathbb{0}_b \tilde{f}_{\varsigma(C)}^{ar(C)}(\mathbb{0}_b \text{len}(\varsigma(C))))$$

To ensure that the oracle queries return binary values to the verifier we append the polynomials  $x_1(x_1 - 1), \dots, x_{\mu}(x_{\mu} - 1)$  to  $Q'$ ; for each response from the oracle the verifier will check the vector has length  $\mu$  and use the  $x_i(x_i - 1)$  to ensure that the oracle has returned data of the correct data type, namely bits.<sup>11</sup>

Next, since a range check may be encoded as an algebraic indexed relation (equation (2)), for all range check conditions in  $R$  we may form a corresponding AIR with a set of defining polynomials  $Q_R$  and global parameters  $\text{gp}''$ . We then combine the two AIRs we have constructed, setting  $Q := Q' \cup Q_R$ , appending the instance-witness data of the range check relation to the former relation  $\text{REL}_{\text{gp}', Q'}$ , and denoting the updated global parameters by  $\text{gp}_1$ .

The claim that  $(\mathbb{x}, w)$  is well-formed if and only if  $C \models_{\varsigma} \phi$ ;  $R$  is direct from Corollary 3.9: well-formed  $(\mathbb{x}, w)$  have integer entries and satisfy the polynomial relations in  $Q$ , whose constraints guarantee that  $C \models_{\varsigma} \phi$  and  $C \models_{\varsigma} R$ . The converse holds by construction.  $\square$

For instances  $((\phi, R), \varsigma)$  to generate valid inputs to Zinc, they must also be well-formed:

**Definition 4.3.** An instance  $((\phi, R), \varsigma) \in \text{REL}_{\text{gp}}^{\text{FOL}}$  is **well-formed** if it transforms via the transform described in Lemma 4.2 to an instance of  $\text{REL}_{\text{gp}, Q}$  for some  $\mathcal{F}$  and  $Q$ , which is well-formed as an algebraic indexed relation.

Lemma 4.2 shows that we may efficiently compile well-formed instance-witness pairs defined in the syntax of FOL (matrix variable symbols and enriched polynomials), into algebraic indexed relation instance-witness pairs which are valid inputs to Zinc-PIOP. Once we have instance-witness pairs of a form which may be provided

<sup>10</sup>Which oracle will later be instantiated with a PCS such as Zip [18, Section 5].

<sup>11</sup>Pedant note: we implicitly appeal to a syntactic category of formal symbols from which to draw the  $x_i$ . We emphasise this because elsewhere in this paper,  $x$  is used as a meta-level variable ranging over values. So for clarity: the  $x$ s-with-subscript written here are formal polynomial variable symbols;  $x$ -without-any-subscript looks similar but is actually a variable ranging over values. Note that including the  $x_i(x_i - 1)$  in  $Q'$  for all  $i \in [ar(C)]$  is equivalent to including the polynomials  $B_{i,v}^{j,x}(B_{i,v}^{j,x} - 1)$  in  $Q'$  for all  $i \in [ar(C)]$ ,  $j \in \{0\} \cup [ar(C)]$ ,  $x \in [\text{len}(\varsigma(C))]$ , and  $v \in [\mu]$ .

<sup>9</sup>More explicitly, the  $\tilde{f}_{\varsigma(C)}^i$  have domain  $[\text{len}(\varsigma(C))]$ , so inputs are of bit length at most  $\lceil \log_2(\text{len}(\varsigma(C))) \rceil = \mu$ , and have range comprising elements of bit length at most  $maxbl$ . To form the composition  $\tilde{f}_{\varsigma(C)}^i(\mathbb{0}_b \tilde{f}_{\varsigma(C)}^j)$ , we require that the length of  $\mathbb{0}_b \tilde{f}_{\varsigma(C)}^j$  is identical to the bit length of inputs to  $\tilde{f}_{\varsigma(C)}^i$ .



as the input to a PIOP (such as Zinc-PIOP), we can commit to the witness data using a PCS and compile the PIOP to obtain a zk-SNARK.

Observe from Figure 4 that  $mkQ_{\mathcal{F}}^x\langle\phi\rangle$  may contain subterms corresponding to  $C_iC_j(X)$ , whose indices depend on evaluations  $f_j(x) = \tilde{f}_{\zeta(C)}^j(\mathbb{0}_b x)$ . Recall further that such  $\tilde{f}_{\zeta(C)}^j$  are ‘pointers’ obtained from interpolating a subset of the rows of  $\zeta(C)$ . Since these values (equivalently, interpolating polynomials) are not public, but rather are part of the witness inside a polynomial commitment, when running a PIOP on the relations constructed in Lemma 4.2 we stipulate that a verifier who verifies that  $\beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x\langle\phi\rangle) = 0$  must first query the polynomial commitments  $\llbracket \tilde{f}_{\zeta(C)}^j(X_1, \dots, X_\mu) \rrbracket$  in  $\mathbb{w}$  for  $\mathbb{0}_b \tilde{f}_{\zeta(C)}^j(\mathbb{0}_b x)$  for all  $j$  for which  $\zeta(C)_j$  acts as a pointer, which values then define  $mkQ_{\mathcal{F}}^x\langle\phi\rangle$ . We assume that the PCS returns the bits of the queried entry of the witness data. Aside from this additional step, our zk-SNARK uses a multilinear PCS to provide input relations to a PIOP which is run in an otherwise standard manner.

**COROLLARY 4.4.** *Let  $\lambda$  denote a security parameter. Suppose  $\phi \in \text{Pred}$  and  $R \in \text{RCheck}$  and  $\zeta$  is an interpretation satisfying  $\zeta(C) \in \mathcal{M}_{ar(C) \times len(\zeta(C))}(\mathbb{N})$ , with  $ar(C), len(\zeta(C)) \in \mathbb{Z}_{\geq 1}$  satisfying  $ar(C) = \text{poly}(\lambda)$ ,  $len(\zeta(C)) = \text{poly}(\lambda)$ . Set  $\mu := \text{maxbl}$ . Set  $\mathcal{F} = \{\tilde{f}_{\zeta(C)}^i \in \mathbb{Z}_\theta^{\mathbb{N}}[X] \mid i \in [ar(C)]\}$  for some  $\theta > 0$ . Then there exists a zk-SNARK to convince a verifier that a well-formed instance  $((\phi, R), \zeta) \in \text{REL}_{\text{gp}}^{\text{FOL}}$  satisfies  $C \models_{\zeta} \phi; R$ , with soundness error  $\varepsilon_{\text{sound}}$  equal to the soundness error of Zinc, instantiated with global parameters  $\text{gp} = (ar(C), 0, 1, (|R| + 2)\mu, \partial)$ .*

**PROOF.** As in Lemma 4.2 we transform the instance  $((\phi, R), \zeta) \in \text{REL}_{\text{gp}}^{\text{EP}}$  into an equivalent instance  $(\mathbb{x}, \mathbb{w}) \in \text{REL}_{\text{gp}, Q}$  which is a valid input to Zinc-PIOP, for some parameters  $\text{gp} = (k, m, n, \mu, \partial)$ . The claim concerning the soundness error then follows from running a zk-SNARK compiled from Zinc-PIOP and Zip. To conclude the corollary, it suffices to compute the global parameters. For the reader’s benefit, we recall the meaning of each such parameter:  $k$  is the number of multilinear polynomials  $f_i$  comprising the witness;  $m$  is the rank of the auxiliary data  $\mathbf{y}$  in the instance;  $n$  is the number of indexing multilinear polynomials  $g_i$  in  $\mathbf{i}$ ;  $\mu$  is the dimension of the hypercube onto which the  $f_i$  and  $g_i$  interpolate; and  $\partial$  is the bit-length bound on the coefficients of the instance-witness data.

Let  $((mkQ_{\mathcal{F}}^x\langle\phi\rangle)_{x \in [len(\zeta(C))]}, R), (\tilde{f}_{\zeta(C)}^i)_{i \in [ar(C)]} \in \text{REL}_{\text{gp}, \mathcal{F}}^{\text{MV}}$  be obtained from the instance  $((\phi, R), \zeta)$ . Following the proof of Lemma 4.2, we then convert these data into an AIR for Zinc. We obtain one instance-witness pair by considering  $(mkQ_{\mathcal{F}}^x\langle\phi\rangle)_{x \in [len(\zeta(C))]}$  and another by considering the range checks  $R$ ; by relabelling indices and defining  $Q, \mathbf{i}, \mathbb{x}$ , and  $\mathbb{w}$  appropriately we can combine these two instances into a single Zinc input relation tuple. Here, however, we (equivalently) consider the two relations side by side, for clarity.

First consider  $\text{REL}_{\text{gp}', Q'}$  where

$$Q' := ((mkQ_{\mathcal{F}}^x\langle\phi\rangle)_{x \in [len(\zeta(C))]}, x_1(x_1 - 1), \dots, x_\mu(x_\mu - 1)),$$

and  $\mathbb{x} = (\llbracket \tilde{f}_{\zeta(C)}^1 \rrbracket, \dots, \llbracket \tilde{f}_{\zeta(C)}^{ar(C)} \rrbracket)$ , and  $\mathbb{w} = (\tilde{f}_{\zeta(C)}^1, \dots, \tilde{f}_{\zeta(C)}^{ar(C)})$ . Here

$$\text{gp}' = (ar(C), 0, 0, \mu, \partial)$$

Next, consider the instances of  $\text{REL}_{\text{gp}'', Q_{\text{Look}}}$  (cf. equation (2)) corresponding to  $R$ ; any single range check  $R_i \in R$  has global parameters

$$\text{gp}_i'' = (1, 0, 1, \log n_a + \log n_t, \partial)$$

for some  $n_a, n_t$ . We next compute  $\log n_a$  and  $\log n_t$ .

Recall  $\log n_a$  is the number of variables on which the multilinear extension comprising the range check witness  $\mathbb{w} = a(X)$  is defined. Since the range checks in  $R$  define range checks on the rows of  $\zeta(C)$ , we may take  $a(X) = \tilde{f}_{\zeta(C)}^i$  for some  $i \in [ar(C)]$  and so each range check in  $R$  comprises at most  $len(\zeta(C))$  integer range checks. We then have that  $n_a$  may be taken to be the minimal integer satisfying  $\lceil \log n_a \rceil = \text{maxbl} = \mu$ . By padding appropriately we may take  $\log n_a = \mu$ .

Recall  $t(Y)$  is the multilinear extension interpolating the set in which the evaluations of  $a(X)$  should lie. Since range checks in our semantics arise to ensure pointers are well-defined, range checks ensure that the entries of some rows of  $\zeta(C)$  lie in  $[len(\zeta(C))]$ . Thus we have  $\log n_t = \mu$  also.

Finally, since  $R$  may contain many range checks, we find that the parameters to perform all the range checks in  $R$  are

$$\text{gp}'' = (|R|, 0, 1, (|R| + 1)\mu, \partial)$$

To compute the final parameters  $\text{gp} = (k, m, n, \mu, \partial)$  of our AIR instance, we note that since the range checks are performed on the same witness data as in the  $\text{REL}_{\text{gp}', Q'}$  instance, we have  $k = ar(C)$ . Moreover, the same choice of  $t(Y)$  can be used for each range check, so  $n = 1$ . Thus the final parameters of the AIR instance are

$$\text{gp} = (ar(C), 0, 1, (|R| + 2)\mu, \partial)$$

□

**Remark 4.5.** The verifier complexity of our zk-SNARK is inherited from the verifier complexity of the zk-SNARK used under the hood of Zinc. The verifier sees a proof, which they verify.

The prover complexity is more complicated. If the input instance is defined on the  $len(\zeta(C))$  polynomials  $\{mkQ_{\mathcal{F}}^x\langle\phi\rangle\}$ , then the prover complexity is directly inherited from the prover complexity of Zinc’s zk-SNARK.

However, if the input instance is defined from an arbitrary FOL predicate, then the prover has to perform the transformations of Figures 2 and 4 to obtain the  $mkQ_{\mathcal{F}}^x\langle\phi\rangle$ . At a high level (this is not a mathematical proof) to compute the enriched polynomial  $\langle\phi\rangle$  from  $\phi$  using Figure 2 requires (assuming no optimisations) to compute a product of sums of squares schema of this form:

$$\langle\phi\rangle = \prod \sum (C_r(X) - C_s(X))^2$$

To compute the polynomials  $mkQ_{\mathcal{F}}^x\langle\phi\rangle$  we replace the  $C_r(X)$  with the  $b2int(B_{r,1}^{j,x}, \dots, B_{r,\text{maxbl}}^{j,x})$  (and likewise the  $C_r C_s(X)$ ) for

$$\prod \sum (b2int(B_{r,1}^{j,x}, \dots, B_{r,\text{maxbl}}^{j,x}) - b2int(B_{s,1}^{j,x}, \dots, B_{s,\text{maxbl}}^{j,x}))^2$$

This can be left in an unexpanded form until one computes the  $\beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x\langle\phi\rangle)$ , at which point the whole polynomial is evaluated at the relevant bit strings.

To the above we should note that this is generic case for an arbitrary predicate  $\phi$ . However, in practice provers do not generate arbitrary  $\phi$ ; they generate  $\phi$  with specific structure reflecting specific meaning. Thus there may be a design space of optimal

syntactic forms for predicates, which optimise proving. This is a familiar pattern: design spaces of optimal syntactic forms are known for other computational applications of logic (e.g. the *Horn clause theories* of logic-programming). It would not be a surprise to find similar structure in this new computational application of logical syntax; investigating this is future work.

*Remark 4.6.* We note that the set  $Q'$  contains  $\text{len}(\zeta(C))$  polynomials  $mkQ_{\mathcal{F}}^x(\phi)$ . Moreover, the equivalence between the initial FOL instance and the instance we construct as an input to Zinc-PIOP holds only if  $\beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x(\phi)) = 0$  for all  $x \in [\text{len}(\zeta(C))]$ . To avoid checking that each and every  $mkQ_{\mathcal{F}}^x(\phi)$  evaluates to zero under  $\beta_{\mathcal{F}}$ , we can run a ‘random linear combinations’ step in which the verifier sends  $\ell_{\zeta}(C) = \text{len}(\zeta(C))$  uniformly random challenges  $c_i$  from a suitable challenge set and the prover proves that the random linear combination

$$\sum_{x=1}^{\ell_{\zeta}(C)} c_x \cdot mkQ_{\mathcal{F}}^x(\phi) \text{ satisfies } \beta_{\mathcal{F}}\left(\sum_{x=1}^{\ell_{\zeta}(C)} c_x \cdot mkQ_{\mathcal{F}}^x(\phi)\right) = 0$$

This comes at the cost of a small increase in the soundness error of the protocol, and the details are beyond the scope of this paper.

We leave as an open question the task of preprocessing the input FOL relations into a logically equivalent form which arithmetises to an AIR of lowest complexity.

## 5 Two examples of arithmetisation in action

*Arithmetising Power Functions.* A common use of zk-SNARKs sees a prover convince a verifier that the prover has computed the output of some function. We now show how a prover can prove knowledge of values of a function expressed in FOL. We consider the power function:

*Definition 5.1.* The standard inductive definition of  $a^b$  for  $a, b \in \mathbb{N}$  is a function  $\text{pow}(x, y)$  satisfying:

$$\begin{array}{ll} \text{base case} & \text{pow}(a, 0) = 1 \\ \text{inductive step} & \text{pow}(a, b+1) = a * \text{pow}(a, b) \end{array}$$

This may be written in first order logic as

$$\forall a. (\text{pow}(a, 0) = 1) \wedge \forall a, b. (\text{pow}(a, b+1) = a * \text{pow}(a, b)),$$

We define the corresponding predicate below, where  $\phi_{\text{pow}}$  is the validity predicate, written in terms of enriched polynomials, giving the condition that  $\text{pow}$  is an arity 4 matrix variable symbol with rows  $\text{pow}_i$  which encodes the power function, and where  $\text{pow}_i(X)$  denotes an enriched polynomial. We let the entries of  $\text{pow}_1$  correspond to bases,  $\text{pow}_2$  correspond to exponents,  $\text{pow}_3$  correspond to the function output, and  $\text{pow}_4$  correspond to pointers. Set

$$\begin{aligned} \text{baseCase}(X) &= (\text{pow}_2(X) = 0) \wedge (\text{pow}_3(X) = 1), \\ \text{indStep}(X) &= \text{pow}_1(X) = \text{pow}_1(\text{pow}_4(X)) \quad \wedge \\ &\quad \text{pow}_2(X) = \text{pow}_2(\text{pow}_4(X)) + 1 \quad \wedge \\ &\quad \text{pow}_3(X) = \text{pow}_1(X) \cdot \text{pow}_3(\text{pow}_4(X)) \end{aligned}$$

Then – using a chained- $\leq$  shorthand for range checks – set

$$(\phi_{\text{pow}}, R_{\text{pow}}) = (\text{baseCase}(X) \vee \text{indStep}(X), 1 \leq \text{pow}_4 \leq \text{len}(\text{pow}))$$

See [15, Lemma 3.2.6] for a proof that  $(\phi_{\text{pow}}, R_{\text{pow}})$  does indeed encode Definition 5.1. We therefore consider instances of the relation

$$\text{Rel}_{\text{gp}}^{\text{FOL}} = \left\{ ((\phi, R), \zeta) \mid \begin{array}{l} \phi = \phi_{\text{pow}}, R = R_{\text{pow}}, C = \text{pow} \\ \zeta(C) \in \mathcal{M}_{4 \times \text{len}(\zeta(C))}(\mathbb{N}) \\ C \models_{\zeta} \phi; R \end{array} \right\}$$

Applying the transform of Figure 2 to  $\phi_{\text{pow}}$ , we obtain an enriched polynomial instance in

$$\text{Rel}_{\text{gp}}^{\text{EP}} = \left\{ ((\langle \phi \rangle, R), \zeta) \mid \begin{array}{l} \phi = \phi_{\text{pow}}, R = R_{\text{pow}}, C = \text{pow} \\ \zeta(C) \in \mathcal{M}_{4 \times \text{len}(\zeta(C))}(\mathbb{N}) \\ C \models_{\zeta} \phi; R \end{array} \right\}$$

$$\begin{aligned} \text{where } \langle \phi_{\text{pow}} \rangle &= (\text{pow}_2(X)^2 + (\text{pow}_3(X) - 1)^2 * \\ &\quad ((\text{pow}_1(X) - \text{pow}_1(\text{pow}_4(X)))^2 \\ &\quad + (\text{pow}_2(X) - (\text{pow}_2(\text{pow}_4(X)) + 1))^2 \\ &\quad + (\text{pow}_3(X) - \text{pow}_1(X) * \text{pow}_3(\text{pow}_4(X))))^2) \end{aligned}$$

We then apply  $mkQ_{\mathcal{F}}^x$  with  $\mathcal{F} = \{(\tilde{f}_{\zeta(\text{pow})}^i)_{i \in [4]}\}$  to obtain an instance in

$$\text{REL}_{\text{gp}, \mathcal{F}}^{\text{MV}} = \left\{ (((mkQ_{\mathcal{F}}^x(\phi))_{x \in [\text{len}(\zeta(C))]}, R), (\tilde{f}_{\zeta(C)}^i)_{i \in [4]}) \mid \begin{array}{l} \phi = \phi_{\text{pow}}, R = R_{\text{pow}}, C = \text{pow} \\ \zeta(C) \in \mathcal{M}_{4 \times \text{len}(\zeta(C))}(\mathbb{N}), C \models_{\zeta} R, \\ \forall x \in [\text{len}(\zeta(C))](\beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x(\phi)) = 0) \end{array} \right\}$$

By Figure 4 the instance polynomials have the form

$$\begin{aligned} mkQ_{\mathcal{F}}^x(\phi_{\text{pow}}) &= (mkQ_{\mathcal{F}}^x(\text{pow}_2(X))^2 + (mkQ_{\mathcal{F}}^x(\text{pow}_3(X) - 1)^2) * \\ &\quad ((mkQ_{\mathcal{F}}^x(\text{pow}_1(X)) - mkQ_{\mathcal{F}}^x(\text{pow}_1(\text{pow}_4(X))))^2 \\ &\quad + (mkQ_{\mathcal{F}}^x(\text{pow}_2(X)) - (mkQ_{\mathcal{F}}^x(\text{pow}_2(\text{pow}_4(X)) + 1))^2 \\ &\quad + (mkQ_{\mathcal{F}}^x(\text{pow}_3(X)) \\ &\quad - mkQ_{\mathcal{F}}^x(\text{pow}_1(X)) * mkQ_{\mathcal{F}}^x(\text{pow}_3(\text{pow}_4(X))))^2) \\ &= (b2int(B_{2,1}^{0,x}, \dots, B_{2,\mu}^{0,x})^2 + (b2int(B_{3,1}^{0,x}, \dots, B_{3,\mu}^{0,x}) - 1)^2) * \\ &\quad \left( (b2int(B_{1,1}^{0,x}, \dots, B_{1,\mu}^{0,x}) - b2int(B_{1,1}^{4,x}, \dots, B_{1,\mu}^{4,x}))^2 \right. \\ &\quad + (b2int(B_{2,1}^{0,x}, \dots, B_{2,\mu}^{0,x}) - (b2int(B_{2,1}^{4,x}, \dots, B_{2,\mu}^{4,x}) + 1))^2 \\ &\quad + (b2int(B_{3,1}^{0,x}, \dots, B_{3,\mu}^{0,x}) \\ &\quad \left. - b2int(B_{1,1}^{0,x}, \dots, B_{1,\mu}^{0,x}) * b2int(B_{3,1}^{4,x}, \dots, B_{3,\mu}^{4,x}))^2 \right) \end{aligned}$$

The condition  $\beta_{\mathcal{F}}(mkQ_{\mathcal{F}}^x(\phi_{\text{pow}})) = 0$  is equivalent to saying that when  $\beta_{\mathcal{F}}$  replaces the  $B_{i,v}^{0,x}$  with  $\pi_v \mathbb{Q}_b \tilde{f}_{\zeta(\text{pow})}^i(\mathbb{Q}_b x)$  and the  $B_{i,v}^{4,x}$  with  $\pi_v \mathbb{Q}_b \tilde{f}_{\zeta(\text{pow})}^i(\mathbb{Q}_b \tilde{f}_{\zeta(\text{pow})}^4(\mathbb{Q}_b x))$ , that is when  $\beta_{\mathcal{F}}$  replaces the  $B_{i,v}^{0,x}$  with  $\pi_v \mathbb{Q}_b \zeta(\text{pow})_{@i,x}$  and the  $B_{i,v}^{4,x}$  with  $\pi_v \mathbb{Q}_b \zeta(\text{pow})_{@i, \zeta(\text{pow})_{@4,x}}$ , we obtain 0. Thus  $mkQ_{\mathcal{F}}^x(\phi_{\text{pow}})$  has roots at the binary decompositions of (a subset of) the entries of  $\zeta(\text{pow})$ , if  $\zeta$  is a valid interpretation.

The final  $\text{REL}_{\text{gp}, Q}$  instance, ignoring range checks for simplicity, has  $Q = (mkQ_{\mathcal{F}}^x(\phi_{\text{pow}}))_{x \in [\text{len}(\zeta(\text{pow}))]}$  and witness  $\mathbb{w} = (\tilde{f}_{\zeta(C)}^i)_{i \in [4]}$ .

*Arithmetising SK Combinator Reduction.* We next consider zk-SNARKs for SK combinator reduction, a Turing-complete system of computation. This allows us to take as an instance any Turing-complete computation, compiled to SK combinator calculus, and run a zk-SNARK on the statement expressed in our FOL semantics to allow a prover to prove knowledge of witnesses to properties of such computations.

The interested reader can find more on SK combinator reduction in [15, Section 3.9]. A nice summary of the benefits of combinator reduction, and how it relates to other computational models, is contained in the early pages of [24]; see also [11]. Furthermore a zk-SNARK-friendly treatment of SK combinators is the proposed EDEN system [1].<sup>12</sup>

We begin by defining the Cantor pairing function:

**Definition 5.2.** Define a **Cantor pairing** map<sup>13</sup>

$$\langle \cdot, \cdot \rangle' : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N} \\ \langle x, y \rangle' = (x + y) * (x + y + 1) + 2 * x$$

We now arithmetise SK combinator reduction. We first define a relation, where  $\phi_{SK}$  is a validity predicate, giving the condition that SK is a matrix variable symbol encoding combinator reduction:

$$\text{Rel}_{\text{gp}}^{\text{FOL}} = \left\{ ((\phi, R), \varsigma) \left| \begin{array}{l} \phi = \phi_{SK}, R = R_{SK}, C = SK \\ \varsigma(C) \in \mathcal{M}_{ar(C) \times len(\varsigma(C))}(\mathbb{N}) \\ C \models_{\varsigma} \phi; R \end{array} \right. \right\}$$

where SK is an arity 8 matrix with rows  $SK_i$  and  $\phi_{SK}$  is defined as follows, where  $SK_i(X)$  denotes an enriched polynomial and  $\langle \cdot, \cdot \rangle := \langle \cdot, \cdot \rangle' + 2$ , where  $\langle \cdot, \cdot \rangle'$  is Cantor pairing. Set

$$\begin{aligned} \text{Kred}(X) &= SK_1(X) = \langle \langle 1, SK_2(X) \rangle, SK_4(X) \rangle, \\ \text{Sred}(X) &= SK_1(X) = \langle \langle \langle 0, SK_3(X) \rangle, SK_4(X) \rangle, SK_5(X) \rangle \quad \wedge \\ &\quad SK_2(X) = \langle \langle SK_3(X), SK_5(X) \rangle, \langle SK_4(X), SK_5(X) \rangle \rangle, \\ \text{Id}(X) &= SK_1(X) = SK_2(X), \\ \text{Par}(X) &= SK_1(X) = \langle SK_1(SK_6(X)), SK_1(SK_7(X)) \rangle \quad \wedge \\ &\quad SK_2(X) = \langle SK_2(SK_6(X)), SK_2(SK_7(X)) \rangle \quad \wedge \\ &\quad SK_8(X) = SK_8(SK_6(X)) + 1 \quad \wedge \\ &\quad SK_8(X) = SK_8(SK_7(X)) + 1, \\ \text{Tran}(X) &= SK_1(X) = SK_1(SK_6(X)) \quad \wedge \\ &\quad SK_2(X) = SK_2(SK_7(X)) \quad \wedge \\ &\quad SK_2(SK_6(X)) = SK_1(SK_7(X)) \quad \wedge \\ &\quad SK_8(X) = SK_8(SK_6(X)) + 1 \quad \wedge \\ &\quad SK_8(X) = SK_8(SK_7(X)) + 1 \end{aligned}$$

<sup>12</sup>To our knowledge [1] is not a peer-reviewed publication. However, if EDEN itself is correct then it is a developed and apparently practical zk-SNARK-friendly treatment of SK combinators (represented as Dyck words injected into strings of finite field elements). It may be worth quoting the authors on why this matters: “Most zkVMs emulate the von Neumann architecture and must prove relations between a program’s execution and its use of Random Access Memory. However, there are conceptually simpler models of computation that are naturally modelled in a zk-SNARK yet are still practical for use.” Amen. With respect to that paper, our ‘zkVM’ is FOL itself, and in this Section we show a simple but direct and effective way to encode SK combinator reduction within it.

<sup>13</sup>The Cantor pairing function *bijets*  $\mathbb{N} \times \mathbb{N}$  with  $\mathbb{N}$  and is equal to  $\langle x, y \rangle' / 2$ . But this would require us to include fractions in our term language, which is a wrinkle which we prefer to avoid for simplicity, since here we just need some *injection* that can be expressed as a polynomial.

Then set

$$(\phi_{SK}, R_{SK}) = (\text{Kred}(X) \vee \text{Sred}(X) \vee \text{Id}(X) \vee \text{Par}(X) \vee \text{Tran}(X), \\ 1 \leq SK_6 \leq len(SK) \wedge 1 \leq SK_7 \leq len(SK))$$

See [15, Lemma 3.9.10] for a proof that  $(\phi_{SK}, R_{SK})$  does in fact encode SK combinator reduction.

Applying the transform of Figure 2 to  $\phi_{SK}$ , we obtain an enriched polynomial instance in

$$\text{Rel}_{\text{gp}}^{\text{EP}} = \left\{ ((\langle \phi \rangle, R), \varsigma) \left| \begin{array}{l} \phi = \phi_{SK}, R = R_{SK}, C = SK \\ \varsigma(C) \in \mathcal{M}_{8 \times len(\varsigma(C))}(\mathbb{N}) \\ C \models_{\varsigma} \phi; R \end{array} \right. \right\}$$

where  $R_{SK}$  contains the range checks  $1 \leq SK_6 \leq len(SK)$  and  $1 \leq SK_7 \leq len(SK)$ , and

$$\begin{aligned} \langle \phi_{SK} \rangle &= (SK_1(X) - \langle \langle 1, SK_2(X) \rangle, SK_4(X) \rangle)^2 \\ &\quad * ((SK_1(X) - \langle \langle \langle 0, SK_3(X) \rangle, SK_4(X) \rangle, SK_5(X) \rangle)^2 \\ &\quad + (SK_2(X) - \langle \langle SK_3(X), SK_5(X) \rangle, \langle SK_4(X), SK_5(X) \rangle \rangle)^2) \\ &\quad * (SK_1(X) - SK_2(X))^2 \\ &\quad * ((SK_1(X) - \langle SK_1(SK_6(X)), SK_1(SK_7(X)) \rangle)^2 \\ &\quad + (SK_2(X) - \langle SK_2(SK_6(X)), SK_2(SK_7(X)) \rangle)^2 \\ &\quad + (SK_8(X) - SK_8(SK_6(X)) - 1)^2 \\ &\quad + (SK_8(X) - SK_8(SK_7(X)) - 1)^2) \\ &\quad * ((SK_1(X) - SK_1(SK_6(X)))^2 \\ &\quad + (SK_2(X) - SK_2(SK_7(X)))^2 \\ &\quad + (SK_2(SK_6(X)) - SK_1(SK_7(X)))^2 \\ &\quad + (SK_8(X) - SK_8(SK_6(X)) - 1)^2 \\ &\quad + (SK_8(X) - SK_8(SK_7(X)) - 1)^2) \end{aligned}$$

using the transform of Figure 2.

Applying  $mkQ_{\mathcal{F}}^x$  to  $\langle \phi_{SK} \rangle$  we obtain the  $len(\varsigma(SK))$  polynomials that (partly) comprise the set  $Q$  for the instance of the relation upon which Zinc-PIOP may be run.

## 6 Conclusion

*Related work: circuit languages and zkVMs.* We discussed related work on arithmetisation in the Introduction; we now continue that discussion with some comment on, and comparison with, circuit languages and zkVMs.

*Circuit languages* (think: Circom, Noir) allow programmers to specify the behaviour of code in a manner suited for subsequent arithmetisation to constraint systems (for example R1CS).

*zkVMs* (think: Cairo, RISC0) fix a machine model for computation, with a cryptographic proof layer on top with which to succinctly or in zero knowledge prove correctness of that computation; so that arithmetisation for VM-based zk-SNARKs takes (*high-level*) *code* to a *circuit language* to *VM execution traces*, which may then be arithmetised to a *constraint system* and finally input to the zk-SNARK of choice.

Examples of zk-SNARKs for such systems include [29, 3, 35]; an example of a combinator-based (rather than von Neumann architecture based) zkVM is in [1].

For comparison, this paper does *not* introduce a new zkVM or circuit-level language for compilation to zk-SNARK inputs. The arithmetised language of this work is logic itself; this operates at a higher level of abstraction.

Now, we consider SK combinators as an example, and any computation performable on a Turing machine could be compiled to SK combinator reduction and then fed into our example FOL-based zk-SNARK. Therefore, we see a risk that a reader might latch onto this computational example and conflate *that example with the entire paper*. For the avoidance of doubt, that would miss the point! This paper gives a compositional arithmetisation of an *expressive general purpose first-order mathematical logic specification language, over finite integer models*.<sup>14</sup> To our knowledge, this is a novel concept.

**Concluding remarks and future work.** We have provided an arithmetisation of an expressive first-order logic over finite models. We can take any predicate expressible in the syntax of Figure 1 and convert knowledge of a valid witness (i.e. interpretation) to an input instance for a zk-SNARK. Thanks to our use of Zinc, we do not have to rewrite these instances to equivalent instances over finite fields (we comment on arithmetisation to finite fields below).

We gave two examples of our arithmetisation, including SK combinator reduction. Since SK combinators are Turing-complete, this shows that our system is powerful enough to convert knowledge of the output of any computation<sup>15</sup> on a Turing machine into zk-SNARK-compatible form, for a zero knowledge proving system.<sup>16</sup>

The maths in this paper bridges the gap between FOL in Figure 1 — a standard and expressive mathematical language — and the concrete polynomials of a zk-SNARK like Zinc. This opens up interesting possibilities for implementation in future work:

- (1) We can insist on cryptographic certification of correct behaviour. ‘Just’ write a FOL predicate describing your desired notion of correctness,<sup>17</sup> and put your black-box (and possibly adversarial) process in a wrapper that insists on a succinct cryptographic proof of correctness, before accepting its proposed output.
- (2) Because FOL itself can directly express computation, it allows us to design computation at the very high level of logic, which in some applications may save us from having to write code targeted at a specific virtual machine or circuit.

<sup>14</sup>Subtlety note: the arithmetisation in this paper is of logical *validity*, not logical *derivability*. This matters and gives our logic significant extra power. For example: it makes it easy for us to include reification as  $\text{reify}(\phi)$  in Figure 1, interpreted ‘for free’ as a no-op  $\langle \text{reify}(\phi) \rangle_{\zeta}^x = \langle \phi \rangle_{\zeta}^x$  in Figure 2. The focus in this paper is on building a zk-SNARK infrastructure and we do not explore the particular FOL language used in this paper for its own sake, but it is in and of itself an extremely interesting logical object.

<sup>15</sup>Pedant point: any *finite* computation, because our witnesses are finite. But this is fine: correctness of finite logic and computation is what we want to cryptographically prove.

<sup>16</sup>It is known to logicians that FOL is powerful, convenient, and versatile. However, non-logicians sometimes hold misconceptions that *FOL* is *logic*, and *logic* is either *undecidable* (which means you can’t use it to decide anything) or *theoretical* (which means you can’t use it to do anything useful) or *not-programming* (which means both of the above *and* has no users). Doing exponentials and SK combinators is a way to formally address these misapprehensions: these two small examples are decidable, practical, programmable, and they have nonempty userbases. Larger examples are also possible, just prolix.

<sup>17</sup>We put ‘just’ in scare quotes here because writing this predicate is the problem of *formal specification*, which is non-trivial. But that is not a bug; it is a business and consulting opportunity.

- (3) Developing on this theme, *transaction correctness*, *compliance rules*, and similar correctness assertions might be natural targets to be expressed in the framework in this paper.

Implementation and benchmarking of the above are natural next steps.<sup>18</sup>

We also plan to build a version of these ideas that works directly over finite fields. This may be particularly appealing to cryptographers, who are used to this datatype.

Such a finite field-based scheme is certainly possible, but we should understand its value in a practical context. Much programming naturally occurs over rational and integer datatypes (e.g. the reader’s bank balance, velocity, and timestamp are natural numbers *not* finite field elements), and using a finite field back end can create a data representation mismatch.

This mismatch can easily be overcome, but the question is where we put the engineering effort and the audit risk of doing so. These are design questions, and various design possibilities exist: Zinc pushes engineering effort onto the cryptographic back end; a finite field version of this work would push the engineering effort more towards the user; another option is to offer a standard abstraction in-between the user and the cryptographic back end.<sup>19</sup>

There is no one right answer here, and the trade-offs may depend on use-case and may also change with time as technology evolves. There is a design space here, for us to explore.

## References

- [1] Logan Allen, Brian Klatt, Philip Quirk, and Yaseen Shaikh. 2023. EDEN - a practical, SNARK-friendly combinator VM and ISA. Cryptology ePrint Archive, Paper 2023/1021. (2023). <https://eprint.iacr.org/2023/1021>.
- [2] S. Angel, E. Ioannidis, E. Margolin, S. Setty, and J. Woods. 2024. Reef: fast succinct Non-Interactive Zero-Knowledge Regex proofs. In *Proceedings of the 33rd USENIX Security Symposium*. USENIX Association, (Aug. 2024), 3801–3818. doi: [10.5555/3698900.3699113](https://doi.org/10.5555/3698900.3699113).
- [3] A. Arun, S. Setty, and J. Thaler. 2024. Jolt: snarks for virtual machines via lookups. In *EUROCRYPT 2024* (LNCS). M. Joye and G. Leander, (Eds.) Vol. 14656. Springer Nature Switzerland, 3–33. doi: [10.1007/978-3-031-58751-1\\_1](https://doi.org/10.1007/978-3-031-58751-1_1).
- [4] Lennart Augustsson. 2024. MicroHs: a small compiler for Haskell. In *Proceedings of the 17th ACM SIGPLAN International Haskell Symposium* (Haskell 2024). Association for Computing Machinery, Milan, Italy, 120–124. ISBN: 9798400711022. doi: [10.1145/3677999.3678280](https://doi.org/10.1145/3677999.3678280).
- [5] J. Barwise. 1977. An introduction to first-order logic. In *Handbook of Mathematical Logic*. Jon Barwise, (Ed.) North Holland, 5–46. ISBN: 0444863885.
- [6] E. Ben-Sasson, A. Chiesa, and N. Spooner. 2016. Interactive oracle proofs. In *TCC 2016* (LNCS). M. Hirt and A. Smith, (Eds.) Vol. 9986. Springer Berlin Heidelberg, 31–60. doi: [10.1007/978-3-662-53644-5\\_2](https://doi.org/10.1007/978-3-662-53644-5_2).
- [7] Eli Ben-Sasson, Alessandro Chiesa, Daniel Genkin, Eran Tromer, and Madars Virza. 2013. SNARKs for C: verifying program executions succinctly and in zero knowledge. In *CRYPTO 2013* (LNCS). Ran Canetti and Juan A. Garay, (Eds.) Vol. 8043. Springer Berlin Heidelberg, 90–108. doi: [10.1007/978-3-642-40084-1\\_6](https://doi.org/10.1007/978-3-642-40084-1_6).
- [8] A. R. Block, Z. Fang, J. Katz, J. Thaler, H. Waldner, and Y. Zhang. 2024. Field-agnostic SNARKs from expand-accumulate codes. In *CRYPTO 2024* (LNCS). L. Reyzin and D. Stebila, (Eds.) Vol. 14929. Springer Nature Switzerland, 276–307. doi: [10.1007/978-3-031-68403-6\\_9](https://doi.org/10.1007/978-3-031-68403-6_9).
- [9] B. Bünz, B. Fisch, and A. Szepieniec. 2020. Transparent SNARKs from dark compilers. In *EUROCRYPT 2020* (LNCS). A. Canteaut and Y. Ishai, (Eds.) Vol. 12105. Springer International Publishing, 677–706. doi: [10.1007/978-3-030-45721-1\\_24](https://doi.org/10.1007/978-3-030-45721-1_24).
- [10] M. Campanelli and M. Hall-Andersen. 2024. Fully succinct arguments over the integers from first principles. Cryptology ePrint Archive, Paper 2024/1548. (2024). <https://eprint.iacr.org/2024/1548>.

<sup>18</sup>A prototype implementation of the core building blocks of our arithmetisation is available at <https://anonymous.4open.science/r/zk-SNARKs-for-First-Order-Logic-2A41/README.md>. It was hand-coded by a human.

<sup>19</sup>Like Cairo’s (un)signed integer libraries: <https://raeskills.io/post/cairo-integers>.

- [11] T. J.W. Clarke, P. J.S. Gladstone, C. D. MacLean, and A. C. Norman. 1980. SKIM - the S, K, I reduction machine. In *Proceedings of the 1980 ACM Conference on LISP and Functional Programming* (LFP '80). Association for Computing Machinery, Stanford University, California, USA, 128–135. doi: [10.1145/800087.802798](https://doi.org/10.1145/800087.802798).
- [12] B. E. Diamond and J. Posen. 2025. Succinct arguments over towers of binary fields. In *EUROCRYPT 2025* (LNCS). S. Fehr and P.-A. Fouque, (Eds.) Vol. 15604. Springer Nature Switzerland, 93–122. doi: [10.1007/978-3-031-91134-7\\_4](https://doi.org/10.1007/978-3-031-91134-7_4).
- [13] W. Ewald. 2019. The Emergence of First-Order Logic. In *The Stanford Encyclopedia of Philosophy*. (Spring 2019 ed.). Edward N. Zalta, (Ed.) Metaphysics Research Lab, Stanford University.
- [14] Murdoch J. Gabbay. 2024. Arithmetisation of computation via polynomial semantics for first-order logic. Cryptology ePrint Archive, Paper 2024/954. (2024). <https://eprint.iacr.org/archive/2024/954/20240627:160807>.
- [15] Murdoch J. Gabbay. 2025. Arithmetising logic: polynomial semantics of FOL. *Journal of Applied Logics*, 12, 6, (Oct. 2025). (direct download). <https://www.collegepublications.co.uk/ifaology/00074>.
- [16] A. Gabizon, Z. J. Williamson, and O. Ciobotaru. 2019. PLONK: permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Paper 2019/953. (2019). <https://eprint.iacr.org/2019/953>.
- [17] Chaya Ganesh, Anca Nitulescu, and Eduardo Soria-Vazquez. 2023. Rinocchio: SNARKs for ring arithmetic. *J. Cryptol.*, 36, 4, (Oct. 2023). doi: [10.1007/s00145-023-09481-3](https://doi.org/10.1007/s00145-023-09481-3).
- [18] A. Garreta, H. Waldner, I. Vlasov, K. Hristova, L. Dall'Ava, M. Čupić, and M. Klein. 2025. Zinc: succinct arguments with small arithmetization overheads from IOPs of proximity to the integers. In *CRYPTO 2025* (LNCS). Y. Tauman Kalai and S. F. Kamara, (Eds.) Vol. 16006. Citations from [18] refer to the full version, available at <https://eprint.iacr.org/2025/316>. Springer Nature Switzerland, 259–291. doi: [10.1007/978-3-032-01907-3\\_9](https://doi.org/10.1007/978-3-032-01907-3_9).
- [19] A. Golovnev, J. Lee, S. Setty, J. Thaler, and R. S. Wahby. 2023. Brakedown: linear-time and field-agnostic SNARKs for R1CS. In *CRYPTO 2023* (LNCS). H. Handschuh and A. Lysyanskaya, (Eds.) Vol. 14082. Springer Nature Switzerland, 193–226. doi: [10.1007/978-3-031-38545-2\\_7](https://doi.org/10.1007/978-3-031-38545-2_7).
- [20] J. Groth. 2016. On the size of pairing-based non-interactive arguments. In *EUROCRYPT 2016* (LNCS). M. Fischlin and J.-S. Coron, (Eds.) Vol. 9666. Springer Berlin Heidelberg, 305–326. doi: [10.1007/978-3-662-49896-5\\_11](https://doi.org/10.1007/978-3-662-49896-5_11).
- [21] K. Jiang, D. Chait-Roth, Z. DeStefano, M. Walfish, and T. Wies. 2023. Less is more: refinement proofs for probabilistic proofs. In *2023 IEEE Symposium on Security and Privacy (SP)*, 1112–1129. doi: [10.1109/SP46215.2023.10179393](https://doi.org/10.1109/SP46215.2023.10179393).
- [22] M. Kohlweiss, M. Pancholi, and A. Takahashi. 2023. How to compile polynomial IOP into simulation-extractable SNARKs: a modular approach. In *Theory of Cryptography* (LNCS). G. Rothblum and H. Wee, (Eds.) Vol. 14371. Springer Nature Switzerland, 486–512. doi: [10.1007/978-3-031-48621-0\\_17](https://doi.org/10.1007/978-3-031-48621-0_17).
- [23] J. Liang, D. Hu, P. Wu, Y. Yang, Q. Shen, and Z. Wu. 2025. SoK: understanding zk-SNARKs: the gap between research and practice. In *Proceedings of the 34th USENIX Security Symposium* Article 108. USENIX Association, USA, 20 pages. doi: [10.5555/3766078.3766186](https://doi.org/10.5555/3766078.3766186).
- [24] J. Nicklisch-Franken and R. Feizerakhmanov. 2024. Massimult: a novel parallel cpu architecture based on combinator reduction. (2024). arXiv: [2412.02765](https://arxiv.org/abs/2412.02765) [cs.DC].
- [25] A. Nitulescu. 2020. Zk-SNARKs: a gentle introduction. (2020). <https://www.dinens.fr/~nitulescu/files/Survey-SNARKs.pdf>.
- [26] M. Orrù, G. Kadianakis, M. Maller, and G. Zaverucha. 2025. Beyond the circuit: how to minimize foreign arithmetic in ZKP circuits. *IACR Communications in Cryptology*, 2, 1, (Apr. 8, 2025). doi: [10.62056/an-4c3c2h](https://doi.org/10.62056/an-4c3c2h).
- [27] B. Parno, J. Howell, C. Gentry, and M. Raykova. 2013. Pinocchio: nearly practical verifiable computation. In *IEEE Symposium on Security and Privacy* 2013. Vol. 59. (May 2013), 238–252. doi: [10.1109/SP.2013.47](https://doi.org/10.1109/SP.2013.47).
- [28] S. Setty, J. Thaler, and R. Wahby. 2023. Customizable constraint systems for succinct arguments. Cryptology ePrint Archive, Paper 2023/552. (2023). <https://eprint.iacr.org/2023/552>.
- [29] S. Setty, J. Thaler, and R. Wahby. 2024. Unlocking the lookup singularity with lasso. In *EUROCRYPT 2024* (LNCS). M. Joye and G. Leander, (Eds.) Vol. 14656. Springer Nature Switzerland, 180–209. doi: [10.1007/978-3-031-58751-1\\_7](https://doi.org/10.1007/978-3-031-58751-1_7).
- [30] J. Thaler. 2022. *Proofs, Arguments, and Zero-Knowledge*. Foundations and Trends® in Privacy and Security. Vol. 4. Now Publishers, (Jan. 2022). doi: [10.1561/9781638281252](https://doi.org/10.1561/9781638281252).
- [31] Urbit Systems Technical Journal. 2018. Nock. <https://nock.is/intro.html>. Accessed 15 April 2026. (2018).
- [32] R. S. Wahby, I. Tzalla, A. Shelat, J. Thaler, and M. Walfish. 2018. Doubly-efficient zkSNARKs without trusted setup. In *2018 IEEE Symposium on Security and Privacy (SP)*, 926–943. doi: [10.1109/SP.2018.00060](https://doi.org/10.1109/SP.2018.00060).
- [33] Y. Wei, X. Zhang, and Y. Deng. 2025. Transparent SNARKs over Galois rings. In *PKC 2025* (LNCS). T. Jager and J. Pan, (Eds.) Vol. 15674. Springer Nature Switzerland, 418–451. doi: [10.1007/978-3-031-91820-9\\_14](https://doi.org/10.1007/978-3-031-91820-9_14).
- [34] Z. Wu, X. Zhang, Y. Deng, Y. Wei, Z. Zhang, and L. Yang. 2025. Polylogarithmic polynomial commitment scheme over Galois rings. In *ESORICS 2025*. Springer-Verlag, 400–420. doi: [10.1007/978-3-032-07891-9\\_21](https://doi.org/10.1007/978-3-032-07891-9_21).
- [35] A. Zapico, V. Buterin, D. Khovratovich, M. Maller, A. Nitulescu, and M. Simkin. 2022. Caulk: lookup arguments in sublinear time. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, (Nov. 2022), 3121–3134. doi: [10.1145/3548606.3560646](https://doi.org/10.1145/3548606.3560646).
- [36] H. Zeilberger, B. Chen, and B. Fisch. 2024. Basefold: efficient field-agnostic polynomial commitment schemes from foldable codes. In *CRYPTO 2024* (LNCS). L. Reyzin and D. Stebila, (Eds.) Vol. 14929. Springer Nature Switzerland, Cham, 138–169. doi: [10.1007/978-3-031-68403-6\\_5](https://doi.org/10.1007/978-3-031-68403-6_5).

## A Open Science

To provide proof-of-concept for this work, we provide an implementation of two key results:

- (1) We provide an implementation of the transform from FOL predicates  $\phi$  to multivariate polynomials  $mkQ_{\mathcal{F}}^x(\phi)$  obtained by composing the transforms of Figure 2 and Figure 4.
- (2) We provide an implementation of the evaluation function  $\beta_{\mathcal{F}}$  on the resulting multivariate polynomials  $mkQ_{\mathcal{F}}^x(\phi)$ , given range-checked witness data.

The implementation of item (1) is given for several examples defined in FOL. These include predicates corresponding to the standard inductive definition of the power function, to a more efficient power function, to factorials, and to Fibonacci numbers. The implementation of item (2) applies to each of the four stated examples.

The code is available at <https://anonymous.4open.science/r/zk-SNARKs-for-First-Order-Logic-2A41/README.md>.

## B Ethical Considerations

The work in this paper is theoretical and mathematical: it does not involve human subjects, personal data, private datasets, deployed systems, live experiments, vulnerability discovery, or malware.

As with other privacy-preserving technologies, it is possible to exploit the ideas in this paper for unethical purposes. Such tools could be incorporated into systems that conceal unlawful conduct, obscure accountability, or frustrate legitimate auditing. Furthermore, incorrect deployment of cryptographic systems can harm honest users: mistakes in implementations, unsound parameter choices, or misunderstood threat models could lead to invalid proofs being accepted, privacy failures, financial loss, or misplaced trust. Additionally, high-level logical specifications may give users an unwarranted sense that a policy or compliance requirement has been completely captured, when the complexity of real-world application prevents this from being the case.

We mitigate these risks by presenting our work as a mathematical construction rather than as a production-ready system. We state our assumptions and the scope of the construction, providing abstract (rather than real-world) examples. We do not disclose vulnerabilities or provide attack procedures against real systems. For these reasons we believe open publication is appropriate.

We encourage any implementation intended for real-world use based on our ideas to undergo independent cryptographic auditing and an application-specific policy review. In particular, applications involving compliance, regulation, or user rights should treat the FOL specification solely as a formal model of a policy and not as a substitute for legal or ethical judgment.